România
Ministerul Apărării Naționale
Academia Tehnică Militară "*Ferdinand I*"

Facultatea de Sisteme Informatice și Securitate Cibernetică
CALCULATOARE ŞI SISTEME INFORMATICE PENTRU APĂRARE ŞI SECURITATE NAŢIONALĂ

Platformă de Analiză Automată a Atacurilor Data Poisoning asupra unei Infrastructuri de Învățare Federate

Coordonator Științific
Cpt. conf. dr. ing. Iulian Aciobăniței

Absolvent
Sd. Sg. Maj. Lepădatu Tudor

Conține _____ file
Inventariat sub numărul _____
Cu poziția din indicator _____
Cu termen de păstrare _____

București
An 2026

Mulțumiri pentru persoanele care au sprijinit procesul de realizare a acestei lucrări

Referatul Coordonatorului Științific

Referatul reprezintă un text în care coordonatorul științific sumarizează, ulterior finalizării conținutului efectiv, efortul pe care l-ați depus și trage concluzii cu privire la gradul de realizare a obiectivelor propuse inițial (cele prezentate în cadrul detalierii). De regulă, acest referat nu depășește cele 4 pagini alocate în acest document.

Declarație de originalitate

Subsemnatul/Subsemnata, **Sd. Sg. Maj. Lepădatu Tudor**, absolvent(ă) al/a **Academia Tehnică Militară "*Ferdinand I*"**, **Facultatea de Sisteme Informatice și Securitate Cibernetică**, declar pe propria răspundere că lucrarea de diplomă cu titlul " **Platformă de Analiză Automată a Atacurilor Data Poisoning asupra unei Infrastructuri de Învățare Federate** " este rezultatul activității mele independente, nu a fost plagiată sau copiată din alte surse și respectă normele de etică academică.

Semnătura,

.....

Tema Proiectului de Diplomă

Tema proiectului de diplomă (sau detalierea) reprezintă un text realizat de coordonatorul științific, în lunile următoare propunerii titlului proiectului de diplomă către facultate, în care detaliază nevoia reală a implementării unui astfel de proiect și modul în care se dorește a fi implementat. În plus, poate prezenta structura pe capitole a viitoarei lucrări, anexele ce vor fi incluse și sursele bibliografice din care studentul se va informa. De regulă, acestă detaliere nu depășește cele 4 pagini alocate în acest document.

Abstract

The Artificial Intelligence integration with tools and applications has evolved since 2020s and the cybersecurity scene tries to adapt frequently. Information security and data integrity is more important than never before, being used by Machine Learning models or Neural Networks trained to perform specific tasks.

From a data science point of view, the quality of information is much important than securing it. The AI evolution has led to the creation of different attack boundaries, from changing the model parameters to perform data poisoning schemes. Confidentiality is the key in maintaining unique aspects for each entity involved in the federated learning process.

In this paper, data poisoning attacks are studied with different options in a segregated simulated infrastructure called federated learning, in which each client may change its scope intentionally or unintentionally. Each simulation has its own configuration providing data scientist with a dedicated environment for testing its machine learning algorithm againts data poisoning attacks.

Cuprins

1 Introducere **1**
1.1 Context . . . 1
1.2 Motivația lucrării . . . 2
1.3 Obiectivele lucrării . . . 2
1.4 Structura lucrării . . . 3

2 Noțiuni Teoretice **4**
2.1 Noțiuni introductive . . . 4
2.1.1 Diferența dintre Machine Learning și Deep Learning . . . 4
2.1.2 Rețea Neuronală . . . 5
2.2 Învățare automată federată . . . 5
2.2.1 Concept . . . 5
2.2.2 Arhitectura FL . . . 5
2.2.3 Procesul de antrenare FL . . . 8
2.2.4 Exemple în viața reală . . . 9
2.3 Atacuri de tip Data Poisoning . . . 10
2.3.1 Definirea tipurilor de atac . . . 10
2.3.2 Vectori de atac . . . 11
2.3.3 Atacul Data poisoning . . . 12
2.3.4 Diferența dintre data poisoning, model poisoning și atacurile backdoor . 12
2.3.5 Impactul poisoning în Federated Learning . . . 13
2.3.6 Mecanisme de protecție rezistente la atacuri *Byzantine* . . . 14
2.3.7 Provocările distribuțiilor Non-IID în Federated Learning . . . 17
2.3.8 Criterii de evaluare a mecanismelor de apărare . . . 17
2.4 Alte Noțiuni . . . 18
2.4.1 Docker . . . 18
2.4.2 Python . . . 19
2.4.3 Rest API . . . 21

3 Proiectare, Implementare și Testare **23**
3.1 Arhitectura Platformei . . . 23
3.2 Implementare Platformei . . . 27
3.2.1 Scenariul complet de utilizare al platformei . . . 30
3.2.2 Structura directoarelor și a fișierelor importante . . . 31
3.2.3 Analiza statică de securitate cu Semgrep . . . 32
3.2.4 Funcții personalizate de agregare și otrăvire . . . 33
3.2.5 Persistența datelor și structura rezultatelor generate . . . 34
3.3 Cerintele Software . . . 35
3.3.1 Cerintele functionale . . . 35
3.3.2 Cerințele nefuncționale . . . 38
3.4 Testare . . . 40
3.4.1 Testarea de Validare a Șabloanelor . . . 40
3.4.2 Testarea creării de funcții personalizate . . . 41
3.4.3 Scenarii de test negative . . . 42
3.4.4 Testarea Pipeline-ului de Simulare . . . 42

3.4.5 Testarea de Integrare 43
3.4.6 Testarea Comunicării WebSocket 43
3.4.7 Testarea Managementului GPU 44
3.4.8 Testarea Fluxului de Anulare 44
3.4.9 Testarea de Securitate 44
3.5 Modele de Analiză 44
3.5.1 Diagrama de Context 45
3.5.2 Diagrama de Componente 46
3.5.3 Diagramele de Flux 46
3.5.4 Diagrama Entitate-Relație (ERD) 48
3.5.5 Diagrama Cazurilor de Utilizare 49

4 Rezultate și Metrici Simulări 51
4.1 Metodologia experimentală 51
4.2 Studiu de Caz 1: Creșterea intensității otrăvirii datelor 51
4.2.1 Detalii practice 51
4.2.2 Rezultate experimentale 52
4.3 Studiu de Caz 2: Cea mai bună protecție pentru un atac specific 52
4.3.1 Detalii practice 53
4.3.2 Rezultate experimentale 53
4.4 Interpretarea rezultatelor 54
4.5 Discuție comparativă cu literatura de specialitate 55
4.6 Limitări și validitatea rezultatelor 56
4.7 Reproductibilitatea experimentelor 56

5 Concluzii și dezvoltare ulterioară 57
5.1 Starea Curentă 57
5.2 Dezvoltare Ulterioară 57
5.2.1 Limitări actuale 57
5.2.2 Direcții de dezvoltare 58

Bibliografie 59

Listă de figuri

2.1 Arhitectura interna a unui dispozitiv 8
2.2 Procedee în învățare automată federată 9

3.1 Medii Python ale platformei 24
3.2 Arhitectura și fluxul platformei 31
3.3 Diagrama de Context — Nivel 0: Marginile Platformei 45
3.4 Diagrama de Context — Nivel 1: Fluxul de date 45
3.5 Diagrama de Componente a Platformei 46
3.6 Diagrama Flux Utilizator 47
3.7 Diagrama Fluxului de Simulare 48
3.8 Diagrama Entitate-Relație (ERD) 49
3.9 Diagrama Cazurilor de Utilizare 50

4.1 Rezultate Experimentale - Cazul 1 52
4.2 Rezultate Experimentale - Cazul 2 54

Listă de Abrevieri

AI . . . . . *Artificial Intelligence* / Inteligență Artificială

API . . . . . *Application Programming Interface*

AST . . . . . *Abstract Syntax Tree*

CPU . . . . . *Central Processing Unit*

CORS . . . . . *Cross-Origin Resource Sharing*

CSV . . . . . *Comma-Separated Values*

CT . . . . . Tomografie computerizată

DL . . . . . *Deep Learning*

ERD . . . . . *Entity-Relationship Diagram*

EU . . . . . *European Union*

F1 . . . . . *F1-score*

FL . . . . . *Federated Learning*

FN . . . . . *False Negative*

GPU . . . . . *Graphics Processing Unit*

HTTP . . . . . *Hypertext Transfer Protocol*

IA . . . . . Inteligență Artificială

IID . . . . . *Independent and Identically Distributed*

IoT . . . . . *Internet of Things*

JSON . . . . . *JavaScript Object Notation*

JWT . . . . . *JSON Web Token*

ML . . . . . *Machine Learning*

Non-IID . . . . . Date neindependente și neidentic distribuite

RCE . . . . . *Remote Code Execution*

REST . . . . . *Representational State Transfer*

RMN . . . . . Rezonanță magnetică nucleară

SIMD . . . . . *Single Instruction, Multiple Data*

SPOF . . . . . *Single Point of Failure*

SRS . . . . . *Software Requirements Specification*

TP . . . . . *True Positive*

TPR . . . . . *True Positive Rate*

Capitolul 1:

Introducere

1.1 Context

Odată cu dezvoltarea sistemelor de calcul moderne și a componentelor hardware, s-au putut realiza produse software complexe cu capacități de stocare net superioare. Revoluția tehnologică a permis nu doar realizarea unor sarcini simple, precum calcule matematice, sau automatizarea unor dispozitive (de exemplu aprinderea automată a unui bec printr-un microcontroler), ci și posibilitatea gestionării mai eficiente a informațiilor digitale (de la date bancare la fișiere media).

Aceasta a devenit treptat principala sursă legitimă de înregistrare a oricărui tip de date (text, imagini, video, audio). Pentru a accesa și actualiza informația digitală, s-au dezvoltat diferite versiuni de baze de date centralizate și distribuite.

Bazele de date centralizate sunt aplicații software specializate ce folosesc resursele sistemului (a stației) pentru a răspunde cât mai rapid interogărilor. Stațiile trebuie să dețină multă putere de stocare și de procesare în comparație cu un sistem de calcul normal destinat utilizatorilor casnici. În altă ordine de idei, s-au dezvoltat și baze de date distribuite, menite să reducă din capacitățile tehnice ale serverului și să stocheze informația sub forma descentralizată. Căutarea resursei în acest context ar presupune interogarea recursivă a fiecărei entități până la găsirea sa. Prin acest mod, nu doar că stațiile pot avea și capabilități tehnice mai reduse, dar și pot păstra copii de rezervă (backup) locale pentru fiecare segment de informație în parte.

Această evoluție naturală spre descentralizare a deschis drumul unor concepte moderne precum învățarea automată federată (federated learning), unde datele nu mai sunt transferate către un server central. În schimb, modelele sunt antrenate local, iar parametrii sunt ulterior agregați global. Astfel, se menține confidențialitatea datelor, fără a compromite performanța modelului.

Evoluția tehnologică continuă a dat naștere la o serie de atacuri cibernetice menite să destabilizeze securitatea aplicațiilor și totodată sustragerea a cât mai multe date sau identități private în contradicție cu normele legale. Cele mai populare atacuri raportate la scară globală pentru anul curent 2025 sunt ransomware (conform [1], în SUA s-au raportat creșteri de 149%), furtul de identitate prin exfiltrarea de credențiale, și phishing. Dezvoltarea modelelor de inteligență artificială a amplificat aceste riscuri, oferind atacatorilor instrumente automatizate pentru generarea și adaptarea atacurilor.

Pentru a limita utilizarea abuzivă a tehnologiilor bazate pe AI, Uniunea Europeană a adoptat în 2024 un set de reglementări stricte privind integrarea acestor module în aplicațiile software, prin AI Act [2].

Progresul din domeniul machine learning și a Large Language Models a fost posibil ca urmare a unui volum masiv de date disponibile și a nevoii tot mai mari de analiză. Acest lucru a determinat apariția unei noi categorii de specialiști, data scientists, dedicați colectării și prelucrării minuțioase a datelor pentru antrenarea modelelor.

Totuși, pe măsură ce investițiile în tehnologii AI au crescut, au apărut și actori rău intenționați care încearcă să exploateze vulnerabilitățile din procesul de antrenare. Întrucât modelele moderne depind de calitatea datelor folosite, acestea au devenit o țintă principală a atacurilor. Atacatorii se regăsesc și ei într-o poziție constantă de adaptare la noile formalități de securitate și încearcă să contracareze fiecare element nou. Astfel, având în vedere complexitatea dezvoltării

unui modul de inteligență artificială specializat pe diferite domenii, ținta s-a redirecționat spre volumul de date pe care acestea le folosesc și care pot determina starea finală a aplicației.

În contextul învățării automate distribuite, literatura de specialitate identifică trei categorii majore de atacuri:

- Atacuri asupra datelor, precum data poisoning, unde setul de antrenare este manipulat pentru a altera comportamentul modelului;

- Atacuri asupra modelului, prin modificarea parametrilor sau a gradientului (de exemplu, model poisoning);

- Atacuri asupra canalului de comunicare, care vizează interceptarea sau modificarea mesajelor dintre entitățile participante.

Lucrarea de față se concentrează pe prima categorie, data poisoning, în cadrul unei infrastructuri de învățare federată.

1.2 Motivația lucrării

Având în vedere aspectele legate de posibilitatea unei intervenții asupra setului de date de antrenare, atac denumit otrăvire a datelor (data poisoning), munca cercetătorilor s-a îngreunat. Preocuparea nu mai este primordial asupra analizei setului de date de antrenare, cât despre menținerea integrității și a confidențialității lor. Pentru a răspunde acestor nevoi, colaborarea dintre cercetători s-a orientat către modele distribuite de lucru, iar învățarea federată (federated learning) a devenit una dintre principalele direcții. Aceasta permite colaborarea între participanți fără a partaja direct seturile lor de date, menținând o barieră naturală împotriva accesului neautorizat. Totuși, deși infrastructura este diferită față de abordările centralizate, vulnerabilitățile rămân, iar atacurile asupra datelor utilizate local pot afecta modelul global.

În urma unei analize proprii, am putut observa diferite soluții/frameworks de simulare a procesului de învățare automată federată, dar fără o integrare cu mecanisme moderne de testare pentru atacuri precum otrăvirea datelor (data poisoning) amintite anterior [3]. Unele dintre aceste framework-uri sunt poate dificil de gestionat și configurat [4], și nu permit extinderea ușoară prin integrare altor componente. În același timp, gândindu-ne la multitudinea de atacuri malware și la platformele de detecție a lor, devine clar că în domeniul inteligenței artificiale lipsește o platformă centralizată, flexibilă, dedicată testării și evaluării cu diferite tipuri de atacuri asupra modelelor distribuite.

Aceste limitări justifică realizarea prezentei lucrări, care își propune dezvoltarea unei platforme de simulare capabile să testeze atacuri de tip data poisoning într-o infrastructură de învățare federată.

1.3 Obiectivele lucrării

Plecând de la neajunsurile prezentate, ne propunem în această lucrare să venim în sprijinul comunității de cercetare științifică în domeniul securizării soluțiilor cu AI cu o platformă de simulare cu sursă deschisă ("open source"), a acestei clase de atacuri pe mai multe direcții. Astfel, oferim cercetătorilor posibilitatea analizei algoritmului de antrenare propriu dezvoltat, plecând de la o rețea neuronală de bază și un set de date uzual (imagini), și testarea sa prin antrenare în diferite condiții. Platforma în sine respectă toate normele unei aplicații software de producție, în care fiecare acțiune are propria sa logică de implementare. Serviciile sunt segregate suficient de mult încât să permită o dezvoltare ulterioară prin integrarea lor cu alte sisteme.

Rezultatele pot fi utile în contextul securizării procesului de antrenare al algoritmului, dar și pentru analiza factorilor de risc la care e expus în acest mediu.

Cercetătorul este cel care furnizează algoritmul Python de antrenare a propriei rețele neuronale sau algoritm de Machine Learning. El setează parametrii simulării atât pentru procesul de antrenare, cât și pentru tipul de atac de otrăvire a datelor. Platforma își propune să simuleze acest tip de atac cu ajutorul acestor setări de început într-un mediu de învățare federată, furnizând la final o comparație între modelul antrenat folosind datele normale de antrenare și cel antrenat cu datele otrăvite. Aceste rezultate pot fi utile în semnalarea unui posibil risc la nivelul modelului dezvoltat, oferind mai apoi soluții de îmbunătățire a implementării sale.

Contribuțiile proprii ale lucrării pot fi sintetizate astfel:

- proiectarea și implementarea unei platforme modulare pentru simularea atacurilor de tip data poisoning în medii de învățare federată;
- integrarea unui flux complet de execuție, de la validarea șablonului și alocarea resurselor GPU până la generarea și persistența rezultatelor;
- extinderea platformei cu suport pentru funcții personalizate de agregare și de otrăvire a datelor, validate sintactic, structural și din perspectiva securității;
- realizarea unor experimente comparative care evidențiază comportamentul mai multor mecanisme de apărare în scenarii Non-IID și sub atacuri de tip label-flipping.

1.4 Structura lucrării

Lucrarea de față este structurată în cinci capitole, după cum urmează:

Capitolul 1, *Introducere*, prezintă contextul actual al evoluției modelelor de inteligență artificială, motivația dezvoltării unei platforme de simulare pentru atacuri în medii federate și definește obiectivele principale ale cercetării.

Capitolul 2, *Noțiuni Teoretice*, introduce conceptele fundamentale de Machine Learning și Deep Learning, detaliază principiile învățării automate federate (Federated Learning) și analizează tipurile de atacuri adversariale, cu accent pe tehnicile de Data Poisoning și particularitățile acestora.

Capitolul 3, *Proiectare, Implementare și Testare*, descrie arhitectura modulară a platformei propuse, fluxul complet al unei simulări de la inițiere până la generarea rezultatelor, cerințele software și tehnologiile utilizate (Docker, Python, FastAPI, React), evidențiind mecanismele de alocare a resurselor GPU și izolare a mediilor de execuție.

Capitolul 4, *Rezultate și Metrici Simulări*, validează viabilitatea platformei prin prezentarea unor studii de caz experimentale. Se analizează comparativ impactul atacurilor asupra modelelor globale și comportamentul diferiților algoritmi de agregare robustă (ex. FoolsGold, Norm Clipping) folosind metrici de evaluare consacrate.

Capitolul 5, *Concluzii și dezvoltare ulterioară*, sintetizează contribuțiile majore ale lucrării și conturează posibile direcții de extindere viitoare, subliniind importanța securizării infrastructurilor distribuite de inteligență artificială.

Capitolul 2:

Noțiuni Teoretice

În acest capitol, vor fi prezentate noțiunile teoretice specifice înțelegerii procesului de dezvoltare a platformei de simulare. Vom începe cu noțiunile introductive despre conceptele de Machine Learning în antiteză cu Deep Learning. În continuare, vom discuta despre învățarea federată și arhitectura unei infrastructuri federate de învățare automată, tipurile de atacuri data poisoning implementate în procesul de simulare a atacurilor, precum și alte noțiuni specifice implementării.

2.1 Noțiuni introductive

Machine Learning și Deep Learning sunt două ramuri importante ale Inteligenței Artificiale care au rolul dezvoltării unor modele specifice rezolvării unor anumite acțiuni. Pornind de la antrenarea de rețele neuronale, ne orientăm atenția spre setul de date de antrenare și spre actorii ce pot interveni în acest proces. Mediul în care testăm oferă o perspectivă reală asupra impactului pe care îl pot avea aceste atacuri la nivelul unei organizații sau aplicații.

2.1.1 Diferența dintre Machine Learning și Deep Learning

Inteligența Artificială (AI) este domeniul vast care înglobează orice tehnică ce permite calculatoarelor să imite comportamentul uman. Informația a evoluat treptat odată cu îmbunătățirea capabilităților de stocare ale dispozitivelor și apariția programelor software complexe. De la simplul format de text, înregistrări audio, până la imagini și video în rezoluții 4K, modul de lucru s-a diversificat constant.

La fel au evoluat și cerințele utilizatorilor, care tind să acceadă către soluții automate care să le rezolve problemele uzuale, precum identificarea de patterns în imagini sau chiar din video, sau generarea de text.

IA vine să rezolve aceste probleme și să introducă algoritmi de rezolvare specifici pentru fiecare tip de informație furnizată.

Machine Learning este o componentă importantă din domeniul IA care se diferențiază de alte metode de antrenare prin optimizările pe care le aduce erorilor ce apar din predicția rezultatului corect. Modelele de ML clasice se bazează pe intervenția umană în factorul de decizie (supervised learning), mai precis datele de intrare sunt etichetate pentru a oferi un context de predicție stabil.

Deep Learning este o subcategorie a Machine Learning, care are rolul de a minimiza intervenția umană și a automatiza procesul de decizie. Prin această metodă se automatizează mare parte din extragerea caracteristicilor pe setul de date, eliminând nevoia de a defini etichete pentru fiecare valoare de intrare (unsupervied learning).

Diferența dintre aceste două concepte este în modul în care acești algoritmi învață și procentul de utilizare a datelor [5]. Scopul principal al învățării automate este predicția. Pe baza unui set de date de antrenare și de testare, se determină o anumită categorie de ieșire predefinită.

2.1.2 Rețea Neuronală

Rețelele Neuronale sunt un subset al Machine Learning și se identifică drept infrastructură de bază din cadrul algoritmilor de Deep Learning. Denumirea de "neuronal" se referă la structura lor internă, în care fiecare caracteristică (feature) este un neuron ce interacționează unii cu alții. Ele sunt compuse din 3 straturi/layers: primul strat îl reprezintă stratul nodurilor de intrare, al doilea strat este denumit "stratul ascuns" (hidden layer) pentru că încapsulează mai multe straturi, iar ultimul strat este cel de ieșire în care se face predicția propriu-zisă. Straturile ascunse sunt concepute pentru a procesa iterativ datele pornind de la starea lor din nodurile de intrare până la stratul de ieșire.

2.2 Învățare automată federată

Evoluția hardware în tehnologie a condus la creșterea numărului dispozitivelor mobile (telefoane, tablete), denumite gadgets datorită faptului că sunt mici, portabile și moderne. Ele au fost mai departe adoptate la scară largă, devenind obiecte indispensabile în era tehnologică ce avea să vină.

2.2.1 Concept

Învățarea automată federată permite lucrul cu modele de ML sau chiar rețele neuronale, antrenate distribuit, pe un număr mare de dispozitive în scopul rezolvării unei probleme de IA. Distribuirea sarcinilor a fost adoptată și în contextul opoziției lucrului centralizat, pe servere ce dețin capabilități hardware performante (plăci grafice de ultimă generație), dar care generează costuri mari și care pot fi predispuse la amenințări de securitate cibernetică, fiind considerate SPOF(Single Point of Failure).

2.2.2 Arhitectura FL

În literatură, există mai multe categorii de arhitecturi de învățare automată federată. În această secțiune ne vom concentra pe clasificarea generală a arhitecturii unei aplicații folosind federated learning, și vom enumera pentru o anumită categorie cum se clasifică dispozitivele utilizate.

Federated Learning, așa cum a mai fost menționat, este organizat dintr-un server (agregator) și multiple dispozitive client. Modul în care aceste entități comunică este fundamentul principal al procesului de îmbunătățire a învățării.

În modul clasic al federated learning, dispozitivele client transmit actualizări ale modelului de bază la un server central care aplică asupra lor o funcție de agregare, reconstruind întreg modelul de bază. Această setare presupune de fapt o delegare a sarcinii de învățare, însă păstrează entitatea centrală necesară îmbunătățirii soluției. Acest fapt nu ține de evitarea posibilității amenințărilor cibernetice (Single Point of Failure), ci doar să ușureze costurile centralizatorului în a procesa local problema, distribuind sarcinile.

Modelul Fully decentralized (peer-to-peer) learning oferă o nouă abordare și rezolvă problema cibernetică amintită. În această setare nu există agregator, îmbunătățirile fiind comunicate între clienți interconectați. Ideea principală se bazează pe înlocuirea comunicării cu agregatorul cu cea dintre dispozitive individuale printr-un protocol prestabilit. În funcție de numărul de dispozitive, se concepe un graf de conexiuni în care fiecare nod reprezintă un client, iar fiecare muchie un canal de comunicație. Restricția principală este ca un dispozitiv să fie conectat la un număr maxim limitat de dispozitive adiacente, prestabilit, în contradicție cu un graf complet

(stea) specific arhitecturii clasice client-server. Nodurile își îmbunătățesc propriile variante ale rețelei și își comunică rezultatele pe care le agregă local, realizând o medie a ponderilor. În comparație cu modelul federated learning clasic, modelul fully decentralized nu specifică de la început dispozitivelor un model de bază global de la care să pornească în procesul de rezolvare a problemei.

	Federated Learning	**Fully Decentralized (peer-to-peer) Learning**
Orchestrare	Un server central de orchestrare organizează antrenarea, dar nu vede niciodată datele brute.	Nu există orchestrare centralizată.
Comunicare pe arie largă	De obicei topologie hub-and-spoke, în care hub-ul reprezintă un furnizor de servicii coordonator (de obicei fără date), iar spoke-urile conectează clienții.	Topologie peer-to-peer, cu un graf de conectivitate posibil dinamic.

Tabel 2.1: Comparație între modelele Federated Learning

Imaginea de mai sus oferă o privire de ansamblu asupra celor două modele de arhitecturi și caracteristicile acestora, pe baza clasificării prezentate în literatura de specialitate [6].

Model	Avantaje	Dezavantaje
Federated learning (centralized coordination)	• mai simplu de configurat (topologie hub-and-spoke) • pornește de la un model global de bază • agregarea centralizată reduce sarcina de calcul pe clienți • necesită mai puține conexiuni (doar client → server) • gestiunea și monitorizarea sunt mai simple	• SPOF (Single Point of Failure) - serverul central • serverul poate deveni țintă pentru atacuri • nu elimină riscurile cibernetice, doar distribuie munca • dependența de coordonator pentru progresul antrenării • necesită infrastructură centralizată permanent disponibilă
Fully decentralized / peer-to-peer learning	• previne atacurile specifice unui server central (evită SPOF) • reziliență crescută - compromiterea unui nod nu destabilizează întregul sistem • îmbunătățirile se propagă prin graf, fără entitate centrală • agregare locală (fiecare nod mediază ponderile) • poate scala natural dacă graful este bine proiectat	• necesită conexiuni suplimentare între clienți • topologie complexă, dificil de administrat • nu există model global inițial furnizat tuturor • performanța depinde de calitatea grafului de conexiuni • nodurile malițioase pot influența direct vecinii • necesită protocoale suplimentare pentru consistența actualizărilor

Tabel 2.2: Compararea modelelor Federated Learning și Fully Decentralized Learning

În tabelul de mai sus, sunt evidențiate avantajele și dezavantajele utilizării celor două tipuri de modele federated learning.

În continuarea acestei lucrări, se va discuta preponderent despre modelul clasic federated learning, fiind unul adoptat la scară largă și care oferă performanțe bune raportat la costurile de producție.

Modelul clasic la rândul său, se cataloghează în literatură în funcție de tipul dispozitivelor care iau parte la procesul de antrenare. Sub acest filtru, există Cross-Device Federated Learning și Cross-Silo Federated Learning.

Cross-Device Federated Learning sunt dispozitivele client IOT uzuale, individuale, care comunică orchestratorului printr-un protocol prestabilit.

Cross-Silo Federated Learning sunt dispozitive din instituții guvernamentale, companii, sau centre de date distribuite geografic. Instituțiile nu doresc să schimbe informații între ele sau cu un furnizor de servicii central, păstrându-și confidentialitatea, folosind federated learning pentru a antrena propriul model pe datele private ale fiecăruia.

2.2.3 Procesul de antrenare FL

Primul pas este stabilirea conexiunii dintre dispozitive și un server de agregare ce permite antrenarea distribuită a tipului de rețea neuronală sau model ML specific problemei. Odată stabilit canalul, în faza de configurare inițială, serverul trimite dispozitivelor starea de bază a rețelei neuronale, ponderile, în vederea antrenării individuale. Fiecare rețea se antrenează cu datele extrase local (on device) și își îmbunătățește configurația interna la fiecare epocă pentru o perioadă de timp bine determinată.

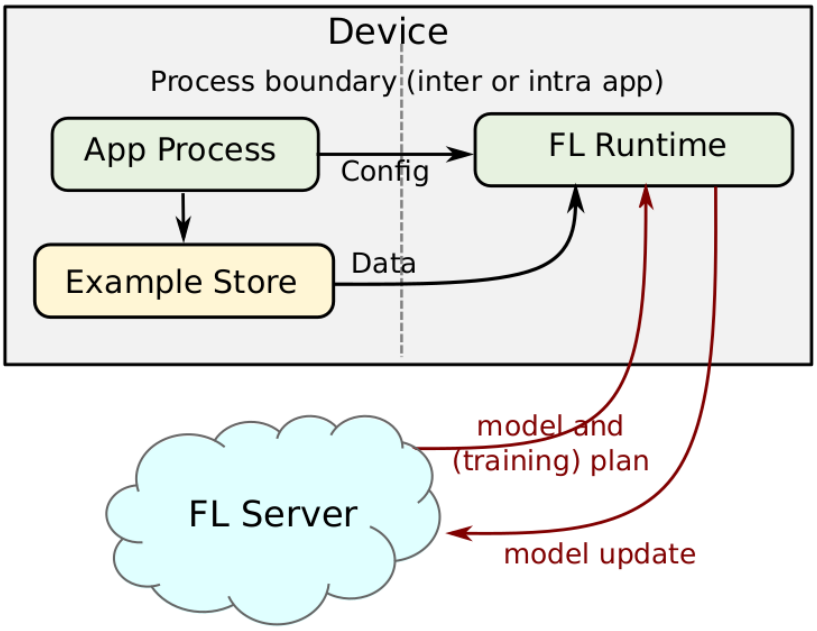

Figura 2.1: Arhitectura interna a unui dispozitiv

Figura de mai sus descrie operațiile specifice programului software care se ocupă de antrenarea rețelei/modelului, conform arhitecturii prezentate de Bonawitz et al. [7]. Putem observa cum dispozitivele primesc un plan de antrenare de bază pe care il vor antrena local pe un set de date limitat.

Deși acest pas nu aduce un procent de îmbunătățiri foarte mari, în faza urmatoare, dispozitivele vor transmite configurațiile curente ale rețelelor lor la orchestrator (server). Rutina FL_Runtime extrage noua configurație locală și îi comunică serverului posibilitatea actualizării acesteia.

În cele din urmă, entitatea centrală combină toate aceste ponderi aplicând o funcție de agregare și în cazul îmbunătățirii setului de ponderi, modifica configurația de bază și o retransmite dispozitivelor pereche. Dacă ponderile noi nu se îmbunătățesc semnificativ fata de configurația de bază, atunci se păstrează aceasta din urma, iar în caz contrar se actualizeaza cu noile ponderi.

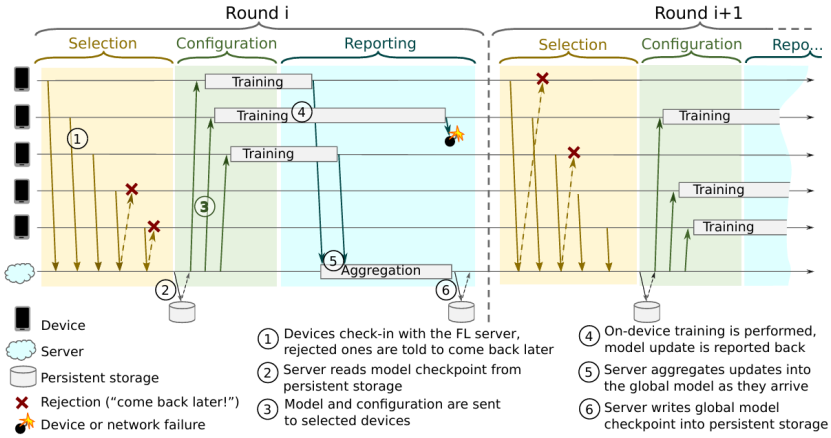

Figura 2.2: Procedee în învățare automată federată

În figura de mai sus, se pot observa într-o manieră continuă, fluxul de comunicație dintre dispozitive și serverul agregator, precum și operațiile specifice fiecărei entități dintr-o rundă de mesaje, într-o formă inspirată din modelul descris în [7].

Securitatea protocoalelor de agregare, utilizate în comunicații dintre clienți și orchestrator, este o componentă importantă în procesul federated learning. De menționat este faptul că, în aceasta topologie, comunicațiile au loc criptat, folosind metode specifice precum criptare homomorfică, sau chiar OTP, însă securitatea datelor de pe dispozitive rămâne la latitudinea acestuia.

2.2.4 Exemple în viața reală

Federated Learning s-a extins rapid în numeroase domenii datorită capacității sale de a antrena modele performante fără a colecta sau centraliza date sensibile. Prin păstrarea informațiilor la nivelul fiecărui dispozitiv sau instituții, FL reduce riscurile asociate scurgerilor de date și permite colaborarea între entități care altfel nu ar putea împărtăși date brute. În continuare sunt prezentate câteva exemple reprezentative ale utilizării sale în aplicații din lumea reală.

Industrie și IoT

- **Mentenanță predictivă:** Vehiculele moderne, utilajele industriale și echipamentele IoT generează constant date despre starea componentelor. FL permite antrenarea unui model comun care poate prezice momentul oportun pentru realizarea mentenanței fără a colecta date brute de la fiecare dispozitiv.

- **Dispozitive de monitorizare:** Senzori purtabili și dispozitive smart home pot furniza statistici privind activitatea sau consumul energetic, păstrând datele utilizatorilor la sursă.

Medical

- **Diagnostic, prognoză și imagistică:** FL este folosit în spitale și clinici pentru detectarea celulelor canceroase din imagini RMN, CT sau radiografii, fără transferul imaginilor către un server central.
- **Confidențialitate menținută la sursă:** Fiecare instituție medicală antrenează local o parte din model, partajând doar actualizările, ceea ce permite colaborarea fără a încălca regulile privind datele pacienților.

Financiar

- **Detectarea fraudelor:** Instituțiile financiare pot îmbunătăți detectarea tranzacțiilor suspecte analizând tipare comune fără a expune date sensibile despre clienți.

Servicii și experiență utilizator

- **Recomandări personalizate:** Platformele de streaming și aplicațiile mobile generează recomandări local, pe dispozitiv, fără a trimite istoricul complet al utilizatorului către server.
- **Analiză comportamentală:** FL poate analiza activitatea utilizatorilor pentru a sugera rutine sănătoase sau îmbunătățiri ale stilului de viață, păstrând confidențialitatea datelor.

Securitate și privacy

- **Supraveghere fără expunerea datelor sensibile:** Modelele de recunoaștere facială pot fi antrenate fără a transmite imagini reale, doar parametrii aferenți.
- **Analiză a sentimentelor:** FL poate analiza reacțiile utilizatorilor la evenimente sociale (like-uri, share-uri, comentarii) fără colectarea directă a acestor date de către platformă.

2.3 Atacuri de tip Data Poisoning

În acest capitol se va discuta unul dintre avantajele pe care le oferă mediul federated learning atacatorilor și ce înseamnă acest lucru pentru fluxul configurației modelului. Vom începe cu definirea vectorilor de atac și concentrarea pe una dintre categorii, data poisoning. În continuarea lucrării, vom analiza impactul pe care îl are acest tip de atac asupra infrastructurii federate de învățare și riscurile pe care le introduce, precum și câteva propuneri de identificare și conștientizare a existenței sale.

2.3.1 Definirea tipurilor de atac

În literatură de specialitate, atacurile asupra unui model de ML sau asupra unei rețele neuronale sunt definite drept atacuri adversariale (adversarial attacks). Această clasă de atacuri are rolul de a produce modificări în comportamentul normal al modelului într-un mod indezirabil. Conform lucrării [6], în funcție de nivelul de scop al unui atac, putem avea:

- **Atacuri nedirectionate (untargeted attacks)** Scopul este reducerea acurateții generale a modelului sau chiar destabilizarea completă a acestuia. Un exemplu pentru clasificarea imaginilor este introducerea unui zgomot care degradează calitatea setului de date. O altă metodă este modificarea etichetelor din setul de antrenare, de exemplu atribuirea etichetei *leu* unor imagini care în mod normal ar trebui etichetate drept *pîșică*.

- **Atacuri directionate (targeted attacks)** Denumite și *backdoor attacks*, deoarece urmăresc modificarea comportamentului modelului doar pentru un anumit subset de date, fără a afecta vizibil acuratețea globală. Continuând exemplul anterior, pentru un anumit tip de imagini ce reprezintă pîșici într-o anumită poziție, se poate introduce un artefact vizual menit să păcălească modelul și să clasifice pîșica drept *leu*. Astfel apare un backdoor activ doar pentru acele imagini precise.

În contextul unor sisteme distribuite care necesită procesarea modelului de ML de o entitate centrală, atacurile adversariale presupun modificarea comportamentului acestui model într-un mod indezirabil. Potrivit acestui scop, principala abordare de compromitere a acestui tip de atac este cea nedirecționată. Aceste atacuri adversariale mai sunt denumite conform cu literatura de specialitate și atacuri *Byzantine* deoarece pot cauza generarea de valori aleatoare nodurilor sistemului distribuit. Plecând de la aceasta categorîșire, există o mulțime de modalitati prin care un atacator poate submina capacitatea de predicție a modelului. Mediul federated learning introduce prin constructie o serie de intrebari la care trebuie găsit un raspuns pentru a determina prin ce moduri un adversar se poate infiltra și poate profita de anumite drepturi pentru a introduce incertitudine în antrenarea sau reglarea (fine-tuning) a modelelor.

2.3.2 Vectori de atac

Analiza amenințărilor asupra modelelor de ML a introdus o serie de posibile vulnerabilități asupra componentelor ce alcătuiesc fluxul de învățare. Un atacator își poate alege zona de interes, pe baza posibilităților de exploatare a sistemului respectiv.

Vectorii de atac cunoscuți sunt:

- **Data Poisoning** Când adversarul încearcă să corupă setul de date de antrenare cu scopul defectării modelului încă de la început. Modificările pot fi subtile sau la o scară largă, urmând ca antrenarea să producă rezultate redundante în viitor.

- **Model Update Poisoning** Când adversarul se folosește de o vulnerabilitate ce îi permite modificarea configurației parametrilor trimiși către orchestrator. Conform cu literatura de specialitate, *Data Poisoning* este considerat a fi o subcategorie de otrăvire a ponderilor modelului, gândindu-ne la impactul pe care antrenarea cu setul de date corupt îl are asupra acestuia.

- **Evasion attack** Când adversarul are acces la datele de testare și le poate modifica în momentul inferenței. Aceasta este o variantă facilă din punctul de vedere al atacatorilor și greu de depistat de țintă, fiind necesare controale speciale asupra integrității setului de testare.

În funcție de gradul de acces la sistemele gazdă, modelul poate fi inspectat în diferite moduri:

- **Black Box** Adversarul nu are abilitatea să inspecteze parametrii modelului înainte sau în timpul atacului.

- **Stale Box** Adversarul poate inspecta doar o versiune incipientă a modelului. Această capabilitate apare și în federated learning când adversarul are acces la rundele de antrenare ale clientului.
- **White Box** Adversarul are abilitatea de a inspecta direct parametrii modelului. Acest scenariu se bazează pe un grad de acces superior al adversarului asupra sistemului.

În funcție de rata de participare a adversarului la atac, putem avea următoarele scenarii:

- În mediul de învățare federată *Cross-device*, atacatorii participa la degradarea modelului doar într-o singura rundă de antrenare. În raport cu procesul de reglare al modelului, un scenariu potrivit ar fi atunci cand setul de date original se mentine actualizat periodic, actorul putand actiona o singura data într-un interval bine stabilit în modificarea acestuia. Impactul în acest sens este aproape nesemnificativ, deoarece fiecare nod client detine o portiune din setul global de date.
- În mediul de învățare federată *Cross-silo*, atacatorii participa la degradarea modelului pentru un numar de runde de antrenare semnificativ apropiat numărului total de runde. Acest aspect introduce, în funcție de gravitatea atacului, o degradare semnificativa și vizibila în timp, cu cât numărul de runde este mai mare.

Pe baza acestor scenarii, atacatorul poate aplică o serie de tehnici pentru modificarea stării modelului. În contextul lucrării de față, se va discuta scenariul stale box, adversarul având posibilitatea doar de a introduce un atac Data Poisoning în rundele de antrenare, datele corupte fiind introduse o singura data în procesul de reglare a modelului (fine-tuning).

2.3.3 Atacul Data poisoning

Acest tip de atac presupune coruperea datelor de antrenare sau de testare, prin diferite tehnici specifice, la nivelul dispozitivului clientului. În aceasta paradigma, atacul poate fi considerat la fel de bine targeted sau untargeted întrucât depinde de intentia adversarului și de potentialul risc în divulgarea punctului de exploatare de care dispune.

În practica atacurilor de tip Data Poisoning, adversarul poate manipula atât conținutul datelor, cât și etichetele acestora, efectele fiind de obicei greu de observat la nivel local. În mediul federated learning această dificultate este amplificată, deoarece orchestratorul nu are acces direct la datele brute ale clienților. Astfel, orice modificare realizată pe un dispozitiv compromis intră automat în procesul de antrenare, fiind tratată ca o contribuție legitimă. În mod particular, chiar și un număr redus de exemple otrăvite poate introduce un comportament persistent în modelul global, mai ales dacă atacul este repetat pe durata mai multor runde

În contextul solutiei propuse, se va discuta despre impactul atacului asupra unui set de imagini și tipurile de metode pentru a le altera, asa cum se poate vedea în tabelul 2.3.3.

Prin aceste tipuri de atacuri adversariale, impactul asupra modelului are loc pe o perioada determinata de timp, de obicei mai lunga, și produce variatii în predicția finala.

2.3.4 Diferența dintre data poisoning, model poisoning și atacurile backdoor

În literatura de specialitate, noțiunile de *data poisoning*, *model poisoning* și *backdoor attacks* sunt strâns legate, dar nu sunt echivalente. Diferențierea lor este importantă deoarece ele descriu fie niveluri diferite de atac, fie obiective diferite urmărite de adversar.

Tip atac	Descriere
Label flip	Modificarea intenționată a etichetelor din setul de antrenare, astfel încât exemple corecte sunt asociate cu clase greșite, afectând procesul de învățare.
Backdoor injection	Inserarea unui artefact vizual sau a unui tipar specific într-un subset mic de date, astfel încât modelul să învețe un comportament anormal activat doar de acel trigger.

Tabel 2.3: Tipuri de atacuri Data Poisoning

Data poisoning reprezintă compromiterea setului de date utilizat la antrenare. Atacatorul modifică imaginile, etichetele sau ambele, astfel încât modelul să învețe din date eronate și să își degradeze performanța sau să dezvolte comportamente malițioase. În acest caz, sursa problemei este chiar informația de intrare folosită la reglarea modelului local.

Model poisoning apare atunci când atacatorul nu mai alterează direct datele, ci manipulează actualizările trimise către serverul de agregare. Acesta poate modifica gradientul, ponderile sau întregul update local, încercând să influențeze direct modelul global. În mediile federate, acest tip de atac este deosebit de periculos deoarece serverul nu are, în mod normal, vizibilitate asupra întregului proces local de antrenare și vede doar rezultatul final transmis de client.

Atacurile backdoor nu descriu neapărat locul unde are loc compromiterea, ci efectul urmărit. Un backdoor este un comportament malițios ascuns, activat de obicei de un trigger specific, în timp ce modelul păstrează o performanță aparent normală pe date curate. Un astfel de atac poate fi implementat atât prin data poisoning, cât și prin model poisoning. Din acest motiv, backdoor-ul trebuie înțeles drept o formă de atac țintit (*targeted attack*), iar nu drept o categorie exclusiv separată la nivel de mecanism.

Relația dintre aceste concepte poate fi rezumată astfel: data poisoning și model poisoning indică *mecanismul prin care adversarul intervine*, iar backdoor attack indică *comportamentul malițios pe care adversarul dorește să îl obțină*. De exemplu, un atac de tip label flip folosit pentru a scădea acuratețea generală este un data poisoning nedirectionat (untargeted), în timp ce inserarea unui trigger vizual pentru a forța clasificarea într-o anumită clasă este un backdoor care poate fi obținut prin data poisoning.

În cadrul prezentei lucrări, accentul este pus pe simularea atacurilor de tip data poisoning, însă efectele observate în experimente pot aparține atât scenariilor nedirectionate (untargeted), cât și celor de tip backdoor. Din acest motiv, analiza mecanismelor de apărare trebuie să urmărească nu doar scăderea acurateții globale, ci și posibilitatea apariției unor comportamente ascunse activate condiționat.

2.3.5 Impactul poisoning în Federated Learning

Mediul de învățare federată a introdus o serie de amenințări cibernetice preponderent la nivelul dispozitivelor clienților, acestea fiind cele mai vulnerabile din punctul de vedere al aplicării unui atac de otrăvire a datelor. Clienții sunt producătorii unui model calitativ care să ofere predicții legitime în diferite scenarii.

Cand se discuta despre alterarea etichetelor (Label Flip Attack) atunci la o prima vedere, utilizatorul nu și-ar da seama decât în urma unei inspectii amanuntite. Pentru acest tip de atac, există tehnici de verificare și filtrare ce pot determina dacă un tip de informație este catalogată corect înainte de antrenare.

Efectele unui data poisoning pot persista chiar și după eliminarea datelor corupte, deoarece

modelul învață un tipar greșit care nu dispare imediat fără o reantrenare completă. Acest lucru este relevant mai ales pentru atacurile backdoor, care rămân inactive până la apariția unui trigger vizual, fără a afecta acuratețea generală. Din acest considerent, atacurile directionate (targeted attacks) sunt deosebit de periculoase în contextul unui mediu de învățare federată pentru ca datele corupte se ascund în interiorul configurațiilor particulare ale clienților. Aceste configurații sunt transmise mai departe la agregator, care, aplicând funcția sa de agregare, amplifică negativ starea modelului.

Având în vedere aceste particularități, devine esențială analizarea metodelor de apărare și a mecanismelor prin care pot fi detectate contribuțiile malițioase. Provocarile în acest domeniu conduc la o serie tot mai mare de utilitare sau platforme ce permit detectarea facila a acestor atacuri și îmbunătățirea evenimentelor cu un potential risc în organizații.

Mediul federated learning introduce un risc suplimentar: un adversar care controlează un număr mic de clienți poate influența disproporționat modelul global dacă este integrat într-un moment critic al antrenării. În absența unor mecanisme robuste de apărare, actualizările malițioase sunt tratate ca fiind legitime, iar agregatorul nu are nicio modalitate directă de a le verifica.

Un alt efect important al acestui tip de atac este degradarea treptată a performanței modelului. În scenariile untargeted, scăderea acurateții globale poate trece neobservată în primele runde de antrenare, dar devine evidentă odată ce modelul converge către o reprezentare eronată a datelor. În scenariile targeted, atacul poate compromite decizii critice doar într-un subset de cazuri, ceea ce face detectarea mult mai dificilă și impactul mult mai nociv, mai ales în aplicații sensibile cum ar fi securitatea, domeniul medical sau sistemele autonome.

2.3.6 Mecanisme de protecție rezistente la atacuri *Byzantine*

În mediul de învățare federată există o multitudine de abordări pentru contracararea atacurilor adversariale, dezvoltate atât din perspectiva securității sistemelor distribuite, cât și din perspectiva robustetii algoritmilor de Machine Learning. Termenul *Byzantine* provine din problema clasică a generalilor bizantini [8], un model teoretic al sistemelor distribuite în care unele noduri pot produce ieșiri arbitrare, fie din cauza unor defecte, fie din cauza unor acțiuni malițioase. În contextul învățării federate, un client bizantin poate trimite actualizări arbitrare către serverul de agregare, cu scopul de a corupe modelul global [6].

Conform lucrării [6], atacurile bizantine reprezintă cazul cel mai defavorabil al atacurilor fără țintă (*untargeted attacks*) asupra unui set de noduri de calcul. Potențialul catastrofal al acestor atacuri a stimulat o linie de cercetare dedicată dezvoltării mecanismelor de agregare rezistente la defecțiuni bizantine [9, 10, 11].

2.3.6.1 Agregare robustă bazată pe estimări statistice

Una dintre cele mai populare categorii de mecanisme de apărare împotriva atacurilor de tip model update poisoning, în special a celor bizantine, constă în înlocuirea funcției de mediere standard (*FedAvg*) de pe server cu o estimare robustă a mediei [6]. Principalele abordări includ:

- **Krum** [9]: selectează actualizarea clientului care prezintă distanța totală minimă față de celelalte actualizări. Prin eliminarea contribuțiilor care se abat semnificativ de la consens, Krum izolează eficient actualizările potențial malițioase. Algoritmul calculează pentru fiecare actualizare suma distanțelor euclidiene față de cele mai apropiate $n-f-2$ actualizări (unde n este numărul total de clienți și f numărul maxim de clienți bizantini) și selectează actualizarea cu suma minimă.

- **Mediana coordonatelor (*Coordinate-wise Median*)** [11]: înlocuiește media aritmetică cu mediana, calculată independent pentru fiecare parametru al modelului. Această abordare este rezistentă la aproximativ 20% clienți malițioși, deoarece mediana nu este influențată de valorile extreme.

- **Trimmed Mean** [11]: elimină un procent predefinit din valorile extreme (cele mai mari și cele mai mici) ale fiecărui parametru, apoi calculează media restului. Această metodă oferă un compromis între robustete și fidelitatea agregării, fiind eficientă împotriva actualizărilor cu magnitudini extreme.

- **FoolsGold** [12]: calculează similaritatea cosinus între actualizările clienților de-a lungul rundelor de antrenare și penalizează contribuțiile cu similaritate ridicată. Această abordare este concepută special pentru a neutraliza atacurile coordonate de tip *sybil*, detectând clienții care trimit actualizări suspect de similare între ei.

- **Norm Clipping**: limitează magnitudinea vectorului de actualizare trimis de fiecare client la un prag maxim prestabilit. Această tehnică previne dominarea procesului de agregare de actualizări cu valori extreme, fiind totodată compatibilă cu mecanismele de confidențialitate diferențială [6].

Studii anterioare au arătat că acești agregatori robuști sunt demonstrabil eficienți pentru învățarea distribuită tolerantă la defecțiuni bizantine [9, 11], sub anumite ipoteze, chiar și în medii federate [6]. Cu toate acestea, Fang et al. [13] au demonstrat recent că mai multe mecanisme de apărare rezistente la atacuri bizantine au oferit o protecție limitată în contextul real al învățării federate. Garanțiile teoretice ale acestor mecanisme se aplică numai sub ipoteze asupra problemei de învățare care de multe ori nu sunt respectate în practică.

2.3.6.2 Provocări specifice mediului federat

Aplicarea directă a mecanismelor de apărare bazate pe agregare robustă în contextul învățării federate introduce o serie de provocări specifice [6]:

- **Datele non-IID (independente și uniform distribuite)**: în scenariile reale de FL, datele disponibile pe fiecare dispozitiv client nu sunt distribuite independent și identic (non-IID), ceea ce înseamnă că actualizările legitime pot prezenta o variabilitate semnificativă. Acest aspect perturbă ipotezele comune din lucrările privind agregarea rezistentă la atacuri bizantine, unde se presupune de obicei o distribuție IID a datelor.

- **Compatibilitate cu agregarea securizată (*Secure Aggregation*)**: protocoalele *SecAgg* asigură confidențialitatea contribuțiilor individuale, dar limitează tipul de operații pe care le poate efectua serverul. Multe metode de agregare robustă existente utilizează operații neliniare pe server, iar compatibilitatea lor cu agregarea securizată nu este încă pe deplin stabilită [6]. Lucrări recente au identificat modalități de a aproxima funcții de agregare precum mediana geometrică sub *SecAgg*, dar soluțiile generale rămân o provocare deschisă.

- **Redundanța și redistribuirea datelor**: o altă linie de apărare împotriva atacurilor bizantine utilizează redundanța seturilor de date și redistribuirea lor pentru a atenua efectele actualizărilor malițioase [10]. Deși aceste mecanisme oferă garanții teoretice riguroase, ele presupun de obicei accesul direct al serverului la date sau posibilitatea de a redistribui global aceste date, condiții care încalcă principiile fundamentale ale FL.

2.3.6.3 Confidențialitatea diferențială ca mecanism de apărare

O abordare complementară agregării robuste este utilizarea confidențialității diferențiale (*differential privacy* – DP) ca mecanism de apărare [6]. Aceasta constă în trei pași: (1) aplicarea unei constrângeri asupra normei ℓ_2 a actualizărilor clienților (*clipping*), (2) agregarea actualizărilor tăiate și (3) adăugarea de zgomot gaussian asupra agregatului. Prin limitarea contribuției oricărui client individual la modelul global, DP previne supraadaptarea la actualizările unui client compromis.

Totuși, DP prezintă limitări importante: zgomotul injectat poate degrada performanța modelului antrenat, iar protecția oferită se degradează exponențial odată cu creșterea numărului de exemple compromise [6]. Mai mult, Wang et al. au demonstrat că atacurile de tip *edge-case backdoor*, generate din mostre cu probabilitate scăzută în distribuția subiacentă, pot ocoli apărările bazate pe DP.

2.3.6.4 Apărarea împotriva atacurilor direcționate

Atacurile direcționate (*targeted attacks*) prezintă provocări distincte față de cele ne-direcționate. Conform [6], chiar și un singur atac efectuat o singură dată poate fi suficient pentru a introduce un *backdoor* într-un model. Bhagoji et al. au arătat că dacă 10% din dispozitivele participante sunt compromise, un *backdoor* poate fi introdus prin otrăvirea actualizărilor trimise serverului, chiar și în prezența detectoarelor de anomalii.

Un aspect deosebit de dificil este faptul că actualizările malițioase ale modelului arată și se comportă în mare parte similar cu modelele antrenate fără atacuri direcționate, evidențiind dificultatea detectării prezenței unui *backdoor*. Cum scopul adversarului este de a afecta rezultatul clasificării doar pentru un subset redus de date, menținând acuratețea globală, mecanismele de apărare concepute pentru atacuri ne-direcționate deseori nu reușesc să contracareze atacurile direcționate [6].

Apărările existente împotriva acestor atacuri necesită fie o examinare atentă a datelor de antrenare, fie acces la un set de validare cu distribuție similară, fie control complet al procesului de antrenare pe server — condiții care nu sunt îndeplinite în mediul FL. Direcții promițătoare includ utilizarea dovezilor cu zero-knowledge (*zero-knowledge proofs*) pentru a garanta că utilizatorii transmit actualizări cu proprietăți prestabilite, sau soluții bazate pe atestare hardware.

2.3.6.5 Atenuarea atacurilor prin sanitizarea datelor

O categorie de apărări concepute specific pentru atacurile de otrăvire a datelor se bazează pe metode de sanitizare (*data sanitization*), care urmăresc eliminarea datelor contaminate sau anormale [6]. Lucrări recente au dezvoltat metode îmbunătățite bazate pe statistici robuste, care oferă avantajul de a fi demonstrabil rezistente la un număr redus de valori aberante.

O clasă conexă de apărări utilizate pentru atacurile *backdoor* sunt cele bazate pe eliminarea selectivă (*pruning*), care încearcă să elimine unitățile de activare inactive pe datele curate. Aceste metode sunt motivate de studii care au arătat că datele contaminate cu scopul introducerii unui *backdoor* activează frecvent anumiți neuroni specifici, denumiți "neuroni backdoor."

Nici sanitizarea datelor, nici eliminarea selectivă nu funcționează direct în mediul FL, deoarece ambele necesită acces la datele clienților sau la date reprezentative similare. Mai mult, studii recente au arătat că multe apărări euristice bazate pe sanitizare rămân vulnerabile la atacuri adaptive de otrăvire, sugerând că abordările directe de sanitizare pot fi insuficiente [6].

Detectarea prezenței datelor otravite, fără identificarea obligatorie a clientului responsabil, rămâne o provocare în FL, amplificată în cazul atacurilor *backdoor*, unde metrici precum

acurateţea globală de antrenare pot să nu fie suficiente pentru a semnala prezenţa atacului.

2.3.7 Provocările distribuţiilor Non-IID în Federated Learning

O caracteristică fundamentală a mediului de învăţare federată este faptul că datele deţinute de clienţi nu respectă, în general, ipoteza clasică IID (*independent and identically distributed*). În practică, fiecare client colectează date în funcţie de propriul context de utilizare, ceea ce înseamnă că distribuţia locală a claselor, volumul de mostre şi chiar caracteristicile semantice ale datelor pot fi diferite de la un nod la altul.

Aceste distribuţii Non-IID pot apărea sub mai multe forme. Unii clienţi pot avea preponderent date dintr-o singură clasă, alţii pot avea foarte puţine exemple, iar alţii pot opera pe un subset de date a cărui distribuţie este foarte diferită de media globală. Din punct de vedere al antrenării, această heterogenitate face ca actualizările legitime ale clienţilor să nu mai fie apropiate între ele în mod natural.

Principala dificultate introdusă de Non-IID este că serverul de agregare trebuie să combine contribuţii care pot părea divergente chiar şi atunci când nu există niciun atac. În consecinţă, separarea comportamentului legitim de cel maliţios devine mai dificilă. O metodă de apărare prea agresivă poate respinge actualizări valide doar pentru că provin dintr-un client cu date foarte diferite, iar o metodă prea permisivă poate accepta mai uşor actualizări compromise.

Din punct de vedere experimental, distribuţiile Non-IID afectează atât viteza de convergenţă, cât şi interpretarea rezultatelor. Modelul global converge mai lent, variaţiile dintre runde cresc, iar degradarea produsă de atac poate fi mai greu de delimitat de variaţia normală a mediului federat. Din acest motiv, experimentele efectuate exclusiv în condiţii IID pot supraestima eficienţa unor mecanisme de apărare.

Pentru platforma propusă în această lucrare, integrarea distribuţiilor Non-IID este esenţială deoarece oferă un cadru mai apropiat de utilizarea reală a federated learning. Evaluarea unui algoritm de agregare robustă în prezenţa heterogenităţii datelor este mai valoroasă decât testarea sa într-un mediu artificial uniform, deoarece reproduce mai fidel dificultăţile întâlnite în practică.

În concluzie, Non-IID nu reprezintă doar o problemă de optimizare, ci şi una de securitate experimentală. Orice analiză riguroasă a atacurilor de otrăvire şi a mecanismelor de apărare trebuie să ţină cont de această caracteristică structurală a mediului federat.

2.3.8 Criterii de evaluare a mecanismelor de apărare

Un mecanism de apărare într-un mediu de învăţare federată nu trebuie evaluat exclusiv prin acurateţea finală a modelului. O apărare poate părea eficientă într-un singur scenariu, dar poate introduce costuri ridicate, instabilitate sau sensibilitate la variaţia parametrilor experimentali. Din acest motiv, evaluarea trebuie realizată după mai multe criterii complementare.

Un prim criteriu este **eficacitatea defensivă**. Aceasta măsoară în ce măsură mecanismul reduce degradarea performanţei sub atac. În scenariile untargeted, acest aspect se observă prin diferenţa dintre performanţa curată şi cea obţinută sub atac. În scenariile targeted sau de tip backdoor, eficacitatea trebuie corelată şi cu blocarea comportamentului maliţios specific, nu doar cu menţinerea acurateţii globale.

Al doilea criteriu este **costul introdus în absenţa atacului**. O metodă robustă poate filtra cu succes contribuţiile suspecte, dar dacă reduce semnificativ performanţa modelului chiar şi atunci când toţi clienţii sunt legitimi, utilitatea sa practică scade. Din acest motiv, comparaţia dintre scenariile Clean şi Clean + Defense este esenţială pentru a evalua overhead-ul apărării.

Un al treilea criteriu este **stabilitatea la variația parametrilor experimentali**. O apărare solidă ar trebui să își păstreze comportamentul și atunci când se modifică procentajul de date compromise, numărul clienților malițioși, distribuția Non-IID sau numărul de runde federate. Metodele care funcționează doar într-o configurație foarte îngustă oferă o valoare practică limitată.

De asemenea, trebuie analizată și **complexitatea computațională**. Unele metode robuste implică operații costîșitoare asupra actualizărilor clienților, ceea ce poate afecta scalabilitatea sistemului. Într-o platformă de simulare, costul în timp și resurse trebuie observat împreună cu beneficiul defensiv adus de algoritm.

Un alt criteriu important este **compatibilitatea cu infrastructură și cu mecanismele de confidențialitate**. Unele tehnici de agregare robustă presupun acces direct la actualizările individuale și aplicarea de operații neliniare, ceea ce poate intra în conflict cu protocoalele de agregare securizată. Astfel, o metodă foarte eficientă teoretic poate fi dificil de integrat într-un sistem real.

Nu în ultimul rând, **interpretabilitatea** reprezintă un criteriu util în analiza experimentală. Este important ca cercetătorul să poată explica de ce un anumit mecanism a funcționat sau a eșuat într-un anumit scenariu. O apărare ușor de interpretat facilitează compararea rezultatelor și construirea unor ipoteze noi de cercetare.

Prin urmare, o evaluare completă a mecanismelor de apărare trebuie să combine performanța sub atac, costul în absența atacului, stabilitatea, scalabilitatea și compatibilitatea operațională. Aceste criterii justifică modul în care au fost structurate studiile experimentale din capitolele următoare.

2.4 Alte Noțiuni

În vederea elaborării soluției propuse în această lucrare, se vor aminti celelalte concepte care stau la baza implementării propriu-zise. În acest capitol se vor detalia succint mecanismele ce stau la baza platformei propuse, modul de utilizare și scopul alegerii lor.

2.4.1 Docker

Docker reprezintă un set de servicii software de tip platformă ce utilizează virtualizarea la nivel de sistem de operare pentru a crea entități independente, numite containere. Aceste containere sunt create specific pentru a întreprinde anumite acțiuni și oferă un mediu izolat de execuție.

Un container este o instanță software ce vine împachetată cu programul aplicației și toate bibliotecile necesare dezvoltării ei. O imagine este vizualizată drept un șablon de instrucțiuni pentru crearea unui container cu un anumit tip de biblioteci necesare dezvoltării unei aplicații. De obicei, imaginile sunt construite pe baza unui fișier denumit Dockerfile care propune o serie de comenzi pentru crearea unui mediu personalizat.

Toate containerele Docker rulează prin intermediul Docker Engine, un serviciu ce rulează la nivelul sistemului de operare și oferă suport cross-platform (Linux, Windows, macOS).

Containerizarea diferă de virtualizarea tradițională prin faptul că containerele partajează același kernel al sistemului de operare gazdă, în timp ce mașinile virtuale (VM) necesită fiecare un sistem de operare complet. Această abordare face containerele Docker mult mai ușoare (de ordinul MB vs GB) și mai rapide la pornire (secunde vs minute) comparativ cu VM-urile.

Arhitectura platformei Docker este asemănătoare modelului client-server, fiind compusă din următoarele componente:

- **Dockerd** un proces daemon, identificabil drept server, ce gestionează tot fluxul de servicii, de la imagini și containere până la volume și rețele.
- **Aplication Binary Interfaces (API)** o suită de interfețe de comunicare și control al serverului
- **Comanda docker** o interfață în linie de comandă, 'docker'

În contextul platformei dezvoltate, Docker oferă un mediu de dezvoltare automat în care se regăsește găzduită platforma publică de simulare. Toate informațiile despre fiecare simulare sunt stocate la nivelul unei baze de date PostgreSQL, în timp ce aplicația web este publică printr-un container frontend, iar în spate regăsim un container backend pentru transmiterea de comenzi serverului.

Comunicația dintre containere are loc în aceeași rețea locală Docker, iar informațiile despre fiecare simulare a clientului persistă în același volum partajat.

Pe lângă Dockerfile, serviciul Docker Engine oferă și posibilitatea creării unei configurații prestabilite pentru definirea de topologii de rețea de containere prin Docker Compose. Pe baza fișierelor Dockerfile în care sunt definite șabloanele imaginilor și a unui fișier de configurare YAML a topologiei relațiilor dintre containere, serviciul Docker Compose permite implementarea unei infrastructuri întregi prin rularea și ștergerea sa dintr-o serie de comenzi.

2.4.2 Python

Python este unul dintre limbajele de programare cele mai des utilizate în contextul global actual. Python este un limbaj interpretat, de nivel înalt, ce vine cu o suită de biblioteci și funcții utile în mai toate domeniile.

Întreaga implementare a operațiunilor backend din proiectul propriu-zis a fost redactată folosind limbajul Python, fiind nu doar util pentru a programa aplicații software, dar și întâlnit frecvent în Machine Learning, pentru antrenarea rețelelor neuronale și a modelelor de învățare automată.

2.4.2.1 Biblioteci Python utilizate

Python a devenit de facto standardul în domeniul ML și Deep Learning datorită ecosistemului său bogat de biblioteci specializate. În contextul inteligenței artificiale, cele mai întâlnite biblioteci utilizate în platforma lucrării sunt:

- **TensorFlow**: una dintre cele mai populare biblioteci open-source pentru Machine Learning, dezvoltată de Google Brain Team. TensorFlow oferă o platformă completă pentru construirea și antrenarea modelelor de Deep Learning, cu suport nativ pentru GPU, ceea ce accelerează semnificativ procesul de antrenare. GPU-urile sunt plăci grafice dedicate procesării unui singur set de instrucțiuni simultan pe multiple seturi de date (SIMD), având integrate mai multe nuclee de procesare decât un procesor normal.
- **PyTorch**: un alt framework major de Deep Learning, dezvoltat de Meta AI Research, cunoscut pentru flexibilitatea sa și abordarea dinamică a grafurilor computaționale
- **scikit-learn**: biblioteca fundamentală pentru Machine Learning tradițional în Python, oferind implementări eficiente ale algoritmilor clasici precum Random Forest, Support Vector Machines, Logistic Regression și k-Nearest Neighbors. În cadrul platformei, scikit-learn joacă un rol crucial prin calculul metricilor de evaluare prin funcții specializate precum confusion_matrix, classification_report, accuracy_score, f1_score.

- **NumPy**: constituie fundația numerică a întregului ecosistem Python pentru calcul științific. NumPy oferă suport pentru array-uri multidimensionale și o colecție vastă de funcții matematice pentru operații vectorizate.

2.4.2.2 Versiuni ale bibliotecilor utilizate

Pentru a asigura reproductibilitatea experimentelor, mediile Python ale platformei au fost configurate cu versiuni explicite ale principalelor biblioteci. Tabelul de mai jos sintetizează versiunile utilizate în backend-ul local, în orchestrator și în cele două medii *Conda* dedicate pentru TensorFlow și PyTorch.

Bibliotecă	**Versiune**	**Rol în platformă**
TensorFlow (`tensorflow[and-cuda]`)	2.15.1	Antrenarea și evaluarea modelelor din mediul `fl_tensorflow`, cu suport pentru accelerare GPU
PyTorch (`torch`)	2.1.0	Antrenarea modelelor din mediul `fl_pytorch` și simularea experimentelor federate pe bază de tensori
torchvision	0.16.0	Transformări și utilitare pentru seturi de date de imagini utilizate în scenariile de antrenare PyTorch
torchaudio	2.1.0	Pachet auxiliar instalat împreună cu ecosistemul PyTorch pentru compatibilitatea completă a mediului
scikit-learn	1.3.2 / 1.3.0	Calculul metricilor de evaluare și generarea rapoartelor de clasificare în mediile TensorFlow, respectiv PyTorch
NumPy	1.26.4 / 1.26.0	Operații numerice vectorizate asupra ponderilor, seturilor de date și rezultatelor intermediare
Pillow	10.4.0 / 10.1.0	Prelucrarea imaginilor și aplicarea transformărilor specifice atacurilor de tip data poisoning
FastAPI	>= 0.110.0	Implementarea backend-ului local și a endpointurilor REST pentru gestionarea simulărilor
Flask	3.0.0 / 3.0.3	Implementarea serviciului orchestrator și a endpointurilor interne de execuție pe serverul de simulare
SQLAlchemy	2.0.25	Persistența proiectelor, fișierelor și rezultatelor simulărilor în baza de date PostgreSQL
websockets	12.0	Transmiterea actualizărilor în timp real privind starea simulărilor către interfața utilizator
tensorflow-datasets	4.9.6	Descărcarea și utilizarea seturilor de date standard în scenariile bazate pe TensorFlow

Tabel 2.4: Versiuni ale principalelor biblioteci Python utilizate în platformă

Prin fixarea explicită a versiunilor s-a urmărit reducerea diferențelor dintre mediile de execuție și prevenirea conflictelor de dependințe care pot afecta rezultatele experimentale sau stabilitatea simulărilor.

2.4.2.3 Utilizarea TensorFlow în cadrul lucrării

În implementarea lucrării, TensorFlow a fost folosit în următoarele scopuri:

- Definirea arhitecturilor de rețele neuronale prin API-ul *Keras*, care oferă o interfață de nivel înalt pentru construirea straturilor (*layers*) și modelelor
- Antrenarea modelelor pe datele locale ale fiecărui client FL prin utilizarea de optimizori, precum *Adam*. Un optimizor este o metodă matematică de reducere a ratei de eroare și de îmbunătățire a acurateții parametrilor modelului de învățare automată.

- Extragerea și setarea ponderilor (*weights*) modelului pentru procesul de agregare, operațiune esențială în FL
- Evaluarea performanței modelului prin metrici precum *accuracy*, *loss*, *precision* și *recall*.

2.4.2.4 Metrici de Evaluare în Machine Learning

Acuratețea (*accuracy*) este definită ca raportul dintre numărul de predicții corecte (atât *true positives*, cât și *true negatives*) și numărul total de predicții:

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total number of predictions}} = \frac{TP+TN}{TP+TN+FP+FN} \tag{2.1}$$

Recall, sau true positive rate (TPR), este metrica ce determină procentul de date care a fost clasificat corect. Această valoare include la numitor atât numărul de elemente a căror predicție a fost identificată corect drept adevărată, cât și numărul elementelor prezise corect drept incorecte.

$$\text{TPR} = \frac{TP}{TP+FN} \tag{2.2}$$

Precizia exprimă proporția predicțiilor pozitive ale modelului care sunt corecte:

$$\text{Precision} = \frac{TP}{TP+FP} \tag{2.3}$$

Anaconda facilitează instalarea și actualizarea rapidă a bibliotecilor necesare, asigurând compatibilitatea între diferite versiuni și reducând riscul apariției conflictelor de dependințe. La nivelul lucrării, au fost create două medii distincte în Anaconda, fiecare configurat cu bibliotecile relevante pentru proiect, TensorFlow și PyTorch, utilizate în procesul de simulare.

Dincolo de capacitățile sale în Machine Learning, Python excelează și în dezvoltarea de aplicații web și servicii backend. În cadrul platformei, Python îndeplinește multiple roluri care vor fi amintite pe tot parcursul capitolului 3 de implementare.

2.4.3 Rest API

REST (Representational State Transfer) API reprezintă o arhitectură software pentru comunicarea între componente distribuite prin protocolul HTTP. Un API (Application Programming Interface) reprezintă o interfață definită special prin care diferite componente software comunică între ele.

În platforma dezvoltată, REST API asigură comunicația între frontend (React), backend (FastAPI) și orchestrator (Flask), implementând un sistem complet de management al simulărilor federate.

2.4.3.1 Arhitectura REST în platformă

Platforma utilizează două servere REST API distincte, fiecare cu responsabilități specifice:

Backend API (FastAPI) rulează în containerul Docker local și servește ca intermediar între interfața utilizator și serverul de simulare. FastAPI este un framework Python modern, de înaltă performanță, bazat pe Pydantic pentru validarea automată a datelor și pe specificația OpenAPI pentru documentare automată. Alegerea FastAPI față de Flask pentru backend este justificată de performanța superioară în scenarii cu cereri concurente și de suportul nativ pentru operații asincrone.

Orchestrator API (Flask) rulează pe serverul ATM și gestionează execuția efectivă a simulărilor. Flask este un framework web minimalist, ideal pentru servicii interne care nu necesită complexitatea FastAPI. Orchestratorul primește comenzi de la backend, alocă resurse GPU, lansează procese de simulare, și returnează rezultatele.

Un aspect fundamental al arhitecturii este comunicarea asincronă între componente. Simulările FL pot dura de la minute la ore, făcând imposibilă așteptarea sincronă a rezultatelor. Platforma implementează pattern-ul Job Queue cu următoarele caracteristici:

- Inițierea simulării se realizează prin cererea POST către backend la endpoint-ul dedicat, stochează configurația în baza de date PostgreSQL, și transmite cererea către orchestrator.
- Monitorizarea progresului se realizează prin două mecanisme complementare: WebSocket pentru actualizări în timp real și polling HTTP pentru backwards compatibility.

Backend-ul expune multiple categorii de endpoint-uri, fiecare responsabil de o funcționalitate specifică:

- Autentificare și Autorizare se realizează prin endpoint-uri dedicate pentru înregistrare utilizatori, autentificare cu username și parolă și generare JWT (JSON Web Token) pentru sesiuni. Token-ul JWT este inclus în header-ul Authorization pentru toate cererile ulterioare, sub forma Bearer token. Backend-ul validează token-ul pentru fiecare cerere protejată, verificând semnătura digitală, expirarea și existența utilizatorului în baza de date.
- Management Proiecte și Fișiere permite utilizatorilor să organizeze simulările în proiecte ierarhice. Fiecare proiect poate conține multiple fișiere reprezentând template-uri Python pentru modele diferite.
- Execuția Simulărilor este orchestrată prin endpoint-ul principal care primește codul template Python și configurația simulării. Backend-ul efectuează validări de securitate asupra codului încărcat pentru a preveni atacurile de tip code injection, verifică existența funcțiilor obligatorii în template și transmite cererea către orchestrator împreună cu token-ul de autentificare.
- Colectarea Rezultatelor se realizează prin endpoint-uri care interogează orchestratorul pentru obținerea rezultatelor finale. După ce orchestratorul semnalează completarea simulării, backend-ul solicită fișierele JSON cu metrici, fișierul text cu rezumatul, și informații despre atacul aplicat.
- WebSocket pentru Monitorizare Real-Time implementează un endpoint special care acceptă conexiuni WebSocket. După stabilirea conexiunii, backend-ul intră într-un loop asincron în care interoghează orchestratorul la fiecare 2 secunde pentru status-ul actualizat al simulării. Informațiile sunt trimise imediat către frontend prin WebSocket, oferind o experiență interactivă similară unui terminal live.

Capitolul 3:

Proiectare, Implementare și Testare

Lucrarea se concentrează pe dezvoltarea unei platforme de simulare a atacurilor de tip Data Poisoning într-un mediu de învățare automată federată. Scopul ei vine chiar din problema identificării acestui tip de atac și îmbunătățirea procesului de alertare în securitatea cibernetică. Platforma sprijină analiștii de securitate și cercetătorii de analiză de date în vederea conștientizării unei posibile vulnerabilități la nivelul unei infrastructuri publice centralizate.

3.1 Arhitectura Platformei

În procesul de dezvoltare al platformei, au fost analizate platforme de programare în timp real, precum Google Colab sau Jupyter Notebook, ce permit utilizatorilor să își încarce și să execute remote codul de antrenare al rețelelor neuronale într-o infrastructură specializată. Inspirându-se din aceste platforme și observând existența unor utilitare de analiză a malware-ului și infecțiilor, s-a identificat nevoia unui framework care nu doar să simuleze o rețea de învățare automată federată, ci și să testeze atacuri Data Poisoning, oferind cercetătorilor și analiștilor de securitate posibilitatea de a evalua vulnerabilitățile propriilor rețele neuronale.

Platforma este structurată pe trei niveluri principale:

Nivelul Client (Frontend)

- Interfață React pentru autentificare, management proiecte și vizualizare rezultate
- Comunicare prin REST API și WebSocket pentru monitorizare în timp-real.

Nivelul Backend (Containerizat Docker)

- API FastAPI pentru gestionarea cererilor utilizatorilor
- Baza de date PostgreSQL pentru persistența configurațiilor și rezultatelor
- Validare securitate cod și transmitere asincronă către orchestrator

Nivelul Execuție (Server ATM)

- Orchestrator Flask pentru managementul simulărilor
- Alocare dinamică GPU prin GPUManager (3 GPU-uri disponibile)
- Medii Python izolate: fl_tensorflow și fl_pytorch

Fluxul platformei este următorul:

- Utilizatorul se autentifică în platformă și își creează un proiect și un fișier de test aferent, în care încarcă un șablon specific pentru antrenare. Șablonul este compus dintr-o serie de funcții predefinite care vor trebui completate cu secvențe de cod specifice creării rețelei neuronale, antrenării și evaluării ei. Printre alte funcții, trebuie să amintim și de cele de setare a ponderilor pentru procesul de agregare la nivelul infrastructurii federate, de descărcare locală a datelor dintr-o sursă specifică și de salvare a configurației rețelei antrenate.

- Odată ce se inițiază simularea la apăsarea butonului Run, șablonul Python este transmis asincron de la backend-ul aplicației către un server remote. Comunicarea asincronă permite utilizatorului să continue lucrul în platformă în timp ce simularea se execută pe server. Serverul remote este denumit Server ATM, fiind localizat în infrastructură Academiei Tehnice Militare "Ferdinand I". Pe serverul remote, cererea este preluată de un proces orchestrator care, odată primită, instanțiază un nou proces care execută o serie de acțiuni.

- Odată inițiată o cerere de simulare, se va crea în directorul utilizatorului autentificat în platformă un director specific, denumit după identificatorul taskului instanțiat de utilizator. În acel director se află toate tipurile de fișiere care vor fi gestionate pe tot parcursul operațiunilor de simulare, de la setul de date de antrenare până la fișierele specifice tipului de atac Data Poisoning utilizat și rezultatele simulărilor în rețeaua federată.

- Prima etapă este cea în care se identifică tipul de biblioteci utilizate pentru procesarea rețelei neuronale și se trece în mediul Python corespunzător. Platforma suportă antrenarea folosind biblioteci TensorFlow și PyTorch, având create câte un mediu specific pentru antrenarea tipului de rețea respectiv. Scriptul de rulare a rețelei neuronale se numește `train_model.py` și este un script agnostic, construit pentru a testa funcțiile din șablon.

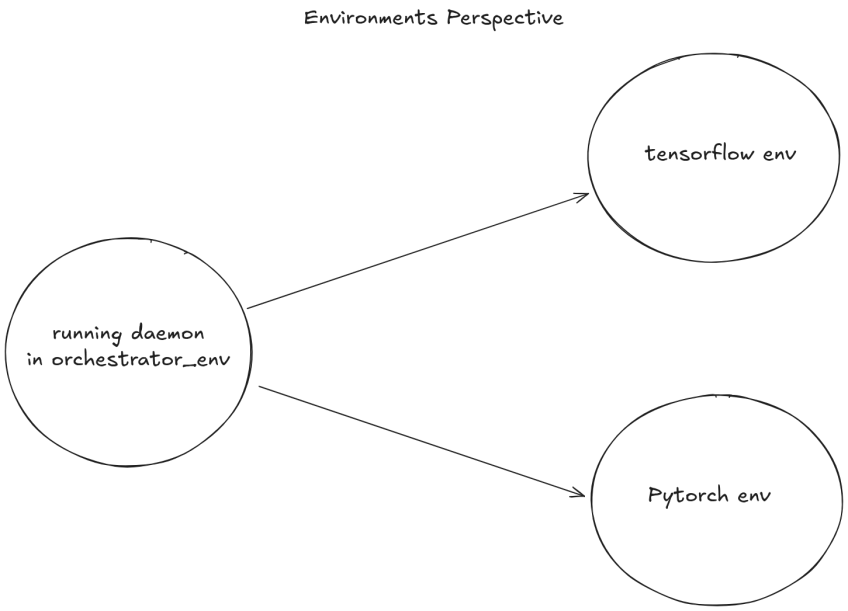

Figura 3.1: Medii Python ale platformei

- Imaginea de mai sus reprezintă mediile Python disponibile pentru rularea rețelelor neuronale.

- La fiecare pas, în cazul unei erori generate de orice fel de acțiune, procesul ce gestionează operațiunile de simulare se oprește și transmite eroarea ca rezultat procesului orchestrator, care notifică backend-ul aplicației.

- După prima antrenare, se salvează parametrii aferenți matricii de confuzie (acuratețe, precizie, scor F1, recall) pentru a compara rezultatul simulării cu configurația inițială a rețelei și detectarea posibilelor anomalii. Acesta este principalul pas în procesul de analiză a problemei propuse.

- Dacă programul este corect structurat, procesul curent încarcă funcțiile scriptului transmis și se folosește de ele pentru a simula atacul Data Poisoning. Mai întâi se apelează funcția de descărcare locală a datelor de antrenare și testare. Aceste date vor fi folosite pentru a simula procesul de otrăvire pe baza configurației simulării setate de utilizator. Datele pot fi descărcate din orice sursă publică, permițând flexibilitate în alegerea tipului de platformă și a datelor pentru testare.

- Argumentele cu ajutorul cărora se rulează scriptul de otrăvire a setului de date sunt:

  - **poison_operation**: tipul atacului de otrăvire a datelor. Platforma suportă 7 tipuri de atacuri:

    * *Label Flip* — inversarea etichetelor unui procent de imagini către o clasă țintă
    * *Backdoor BadNets* — inserarea unui trigger vizual mic (cruce, pătrat) pe imagini
    * *Backdoor Blended* — combinarea imaginii originale cu un pattern cheie folosind un raport configurabil
    * *Backdoor Sinusoidal* — adăugarea unui semnal sinusoidal cu frecvență și amplitudine configurabile
    * *Backdoor Trojan* — inserarea unui watermark în imagini
    * *Backdoor Semantic* — modificarea caracteristicilor naturale ale imaginii (luminozitate, tonalitate RGB)
    * *Edge-Case Backdoor* — aplicarea de transformări asupra imaginilor (rotație, reducere culori, solarizare)

  - **poison_intensity**: intensitatea atacului folosit doar în cazul atacurilor backdoor (ex: 0.1 pentru zgomot Gaussian, dimensiune trigger pentru backdoor)

  - **poison_percentage**: procentajul de date afectate per client (ex: 0.2 pentru 20% date compromise la nivelul unui client malițios)

- Amintim că mediile Python includ și biblioteci de seturi de date specifice, precum TensorFlow Datasets, în care se regăsesc seturi de date predefinite, în cazul în care utilizatorul dorește să folosească un astfel de set de date la nivel de biblioteci.

- Procesul de otrăvire (poisoning) presupune rularea unui script care pe baza parametrilor simulării, aplică o serie de operații asupra seturilor de date. Tipurile de acțiuni de otrăvire au fost definite în capitolul 2 (secțiunea 2.3.3 - Atacul Data poisoning). Utilizatorul poate alege procentajul din etichete care să fie infectate, iar în cazul atacurilor backdoor, se vor seta parametrii specifici fiecărui tip de trigger.

- În pașii următori, se va realiza simularea propriu-zisă cu ajutorul unui script agnostic denumit fl_simulator, care va primi drept argumente următoarele:

  - N numărul de clienți/dispozitive aferente rețelei federate
  - M numărul de clienți compromiși din totalul de mai sus
  - R numărul total de runde de antrenare
  - ROUNDS numărul de runde în care dispozitivele compromise vor antrena cu datele corupte
  - strategia de propagare a dispozitivelor compromise:

    * *first*: dispozitivele compromise vor rula primele ROUNDS runde cu datele otrăvite, iar restul de R-ROUNDS runde vor folosi datele originale
    * *last*: dispozitivele compromise vor rula ultimele ROUNDS runde cu datele otrăvite, iar restul de R-ROUNDS runde vor folosi datele originale
    * *alternate*: dispozitivele compromise vor rula câte o rundă folosind datele otrăvite și câte una cu datele originale
  - metoda de agregare robustă, selectată de utilizator:
    * *FedAvg* — media ponderată a actualizărilor clienților, proporțională cu dimensiunea setului de date
    * *Krum* — selectează actualizarea clientului cu distanța totală minimă față de celelalte, izolând contribuțiile malițioase
    * *Trimmed Mean* — elimină primele și ultimele 20% valori extreme ale ponderilor, apoi calculează media restului
    * *Median* — înlocuiește media cu mediana per parametru, fiind rezistentă la aproximativ 20% clienți malițioși
    * *Trimmed Mean + Krum* — abordare hibridă care combină filtrarea Trimmed Mean cu selecția Krum
    * *FoolsGold* [12] — calculează similaritatea cosinus între actualizările clienților de-a lungul procesului de antrenare și penalizează (reduce ponderile de agregare) pentru contribuțiile cu o similaritate ridicată, reușind să neutralizeze eficient atacurile coordonate (de tip sybil)
    * *Norm Clipping* — limitează magnitudinea (norma L2) a vectorului de actualizare trimis de un client la un prag maxim prestabilit, oprind astfel actualizările malițioase cu valori extreme din a domina procesul de agregare
    * *Random* — selectează aleator ponderile unui client, utilizat ca bază de comparație

- Simularea propriu-zisă execută **patru scenarii distincte**, fiecare pornind de la aceeași configurație inițială a modelului pentru a asigura comparabilitatea rezultatelor:

  1. **Simulare cu date curate (Clean)** — toate dispozitivele folosesc date nemodificate, iar agregarea se face cu FedAvg. Acest scenariu stabilește performanța de referință (baseline) în condiții ideale, fără atac.
  2. **Simulare cu date curate și apărare (Clean + Defense)** — datele rămân nemodificate, dar se aplică metoda de agregare robustă selectată de utilizator. Acest scenariu măsoară costul computațional (overhead-ul) introdus de mecanismul de apărare în absența unui atac.
  3. **Simulare cu date otrăvite (Poisoned)** — M din N clienți folosesc date otrăvite, iar agregarea folosește FedAvg (fără apărare). Acest scenariu cuantifică impactul atacului data poisoning asupra modelului global.
  4. **Simulare cu date otrăvite și apărare (Poisoned + Defense)** — același scenariu de atac, dar cu agregarea robustă activată. Comparând rezultatele cu scenariul anterior, se evaluează eficacitatea strategiei de apărare selectate.

- În urma celor patru simulări, se vor salva informațiile cu privire la fiecare configurație simulată, ce va fi utilizată în procesul de analiză.

  Analiza rezultatelor presupune compararea următoarelor metrici între cele patru scenarii:

- **Acuratețea** — valorile de acurateț pentru fiecare scenariu (Inițial, Clean, Clean+Defense, Poisoned, Poisoned+Defense)
- **Degradarea acurateții** — diferențele între scenarii pentru cuantificarea impactului atacului și a eficacității apărării:
  * Degradare absolută: $\Delta_{acc}^{abs} = Acc_{clean} - Acc_{poisoned}$
  * Variații față de modelul inițial: diferența dintre modelul final din fiecare scenariu și acuratețea pre-antrenată de referință (ex. $Acc_{clean} - Acc_{init}$, $Acc_{poisoned} - Acc_{init}$)
- **Metrici de clasificare**: Precision, recall și F1-score (medie ponderată) pentru fiecare scenariu

Rezultatele sunt salvate în trei fișiere: `analysis.json` (metrici structurate), `summary.txt` (rezumat text asupra simulărilor) și `attack_info.json` (detalii despre atac).

- Utilizatorul va putea vizualiza fiecare pas de procesare amintit până acum și rezultatul analizei simulării.
- Pentru a facilita munca analistului, platforma include posibilitatea de export a raportului simulării în format PDF și CSV, dar și compararea de simulări ale aceluiași proiect.

3.2 Implementare Platformei

Platforma este compusă din două servere, unul dintre ele fiind remote și dispunând de trei plăci grafice NVIDIA A40 (GPU) optimizate pentru medii de lucru cu Inteligență Artificială și Deep Learning. Plăcile dispun de 46 GB memorie VRAM (video RAM) GDDR6 pentru a suporta lucrul intensiv cu date. A40 este o placă profesională destinată centrelor de date, furnizând accelerare hardware pentru TensorFlow și PyTorch, precum și pentru operații matematice complexe necesare în procesele de simulare și învățare federată.

Serverul care rulează funcționalitățile backend și de persistență se află în rețeaua locală proprie. Acesta este responsabil pentru gestionarea interacțiunii utilizatorilor, stocarea configurațiilor și transmiterea cererilor către serverul remote. El reprezintă punctul de intrare al platformei pentru toți utilizatorii și menține logica de securitate, autentificare și gestionare a sesiunilor.

Pentru scalabilitate și flexibilitate, s-au utilizat containere Docker la nivelul serverului local pentru a stoca următoarele componente:

- **Containerul frontend** - include toate dependințele necesare pentru rularea unei aplicații React folosind un server NodeJs. Acesta oferă o interfață modernă și receptivă, permițând utilizatorilor accesarea tuturor funcțiilor platformei, precum încărcarea de șabloane, vizualizarea rezultatelor și compararea simulărilor.
- **Containerul bazei de date PostgreSQL** - stochează toate fișierele cu rezultatele simulărilor executate pe server, specifice fiecărui utilizator, precum și parolele hash ale utilizatorilor. Structura bazei de date este optimizată pentru volum mare de date și interogări rapide asupra istoricului de simulări.
- **Containerul backend** - procesul responsabil de primirea cererilor API. Backend-ul recepționează comenzi de la utilizatori sub forma unei cereri POST către endpoint-ul `/run`. După validare, cererea este retransmisă asincron către serverul remote. Backend-ul implementează

validări de securitate asupra șabloanelor, logarea fiecărei cereri și mecanisme de prevenire a supraîncărcării serverului remote.

La nivelul serverului remote, platforma include următoarele componente:

- **Procesul orchestrator** - rulează scriptul `orchestrator_gpu.py`, care creează un proces nou pentru fiecare conexiune inițiată. Pentru fiecare simulare se alocă un GPU, asigurând un flux de procesare ridicat fără a afecta timpul de execuție al simulărilor deja active. Orchestratorul gestionează coada de task-uri, starea proceselor și comunicarea bidirecțională cu serverul local. Pipeline-ul de simulare parcurge 14 etape secvențiale, de la verificarea structurii șablonului, alocarea resurselor și descărcarea datelor, până la execuția celor patru scenarii de simulare și generarea rezultatelor finale.
- **Managerul de GPU** - implementat în scriptul `gpu_manager.py`, monitorizează în timp real starea GPU-urilor (nivel de utilizare, memorie liberă, temperatură) și distribuie simulările în mod echilibrat pentru a preveni supraîncărcarea unuia dintre dispozitive. Fiecare simulare solicită un GPU dedicat cu un timeout de 10 minute; dacă niciunul nu este disponibil în acest interval, simularea trece automat pe CPU. GPU-ul este **garantat eliberat** la finalizarea, anularea sau eșecul simulării.
- **Procesul de simulare a atacului** - identifică tipul bibliotecilor Python folosite în șablonul primit și comută automat în mediul Anaconda corespunzător (TensorFlow sau PyTorch). Mediile Python sunt complet izolate, având propriile dependințe pentru a asigura reproducibilitatea rezultatelor. Otrăvirea datelor este realizată prin scriptul `poison_data.py`, care suportă 7 tipuri de atacuri cu parametri configurabili per atac. Simularea propriu-zisă rulează prin `fd_simulator.py`, care utilizează șablonul de cod stocat în `template_code.py`. În timpul execuției, fiecare etapă este jurnalizată, iar eventualele erori sunt transmise backend-ului în timp real.
- **Fluxul de anulare** - la fiecare etapă din pipeline, orchestratorul verifică dacă utilizatorul a solicitat anularea. În caz afirmativ: arborele de procese este terminat (`kill_process_tree()`), GPU-ul alocat este eliberat, directorul de lucru al simulării este șters, iar statusul este marcat ca *anulat*.

Etapele procesului de simulare

Atunci când un utilizator inițiază o simulare de învățare federată, orchestratorul de pe serverul GPU declanșează un pipeline secvențial compus din 14 etape distincte:

1. **Pornirea task-ului de simulare (Start Simulation Task)**: Orchestratorul creează o nouă intrare pentru task și începe execuția asincronă a pipeline-ului.
2. **Verificarea structurii șablonului (Verify Template Structure)**: Înainte de a consuma resurse computaționale (cum ar fi GPU), se efectuează o analiză statică suplimentară a scriptului Python de antrenare furnizat de utilizator, folosind modulul intern `verify_template.py`.
3. **Crearea directorului de lucru (Create User Workspace)**: Este creat un director izolat pe disc, specific task-ului curent, pe baza ID-ului utilizatorului și a identificatorului unic al simulării.

4. **Verificarea corectitudinii modelului (Verify Model Correctness)**: Utilizând codul șablon primit, orchestratorul instanțiază modelul definit de utilizator și rulează o scurtă încercare de antrenare pentru a valida funcționalitatea acestuia local, înainte de modelul federativ.

5. **Descărcarea modelului din repozitoriu (Download Model)**: Se apelează logica de definire a modelului din șablon; dacă modelul este pre-antrenat și găzduit extern (ex. HuggingFace), acesta este descărcat și salvat de pe server în formatul specific framework-ului detectat.

6. **Descărcarea setului de date (Download Dataset)**: Orchestratorul apelează funcția de descărcare și pregătire a datelor (testare și antrenare) propusă în codul utilizatorului.

7. **Alocarea unității de procesare grafică (Allocate GPU)**: Managerul de GPU caută o placă video NVIDIA disponibilă în sistem, procesând elementele dintr-o coadă de așteptare. Dacă timpul alocat expiră, se utilizează unitatea CPU.

8. **Activarea mediului Python (Activate Python Environment)**: Prin analiza importurilor din șablon, orchestratorul detectează tipul aplicației (TensorFlow sau PyTorch) și activează mediul *Conda* preconfigurat corespunzător.

9. **Alterarea datelor de antrenare (Poison Dataset)**: Folosind scriptul `poison_data.py`, orchestratorul creează o copie malformată a datelor destinate antrenării, aplicând exact tipul de atac, procentajul și intensitatea alese din interfață.

10. **Simularea în mediul curat (Clean Simulation)**: Modelul global intră în faza de simulare federată (folosind `fd_simulator.py`). Procesul asincron este inițiat exclusiv pe setul de date nealterate (curat), partajat pe N clienți legitimi, folosind algoritmul standard de agregare, *FedAvg*. Această etapă înregistrează performanța normală (de referință).

11. **Simularea curată cu mecanism de apărare (Clean + Defense)**: Continuându-se procesele pentru testare, *FedAvg* este înlocuit acum cu tehnica robustă de agregare aleasă de utilizator (ex. FoolsGold, Norm Clipping), tot pe setul curat de date.

12. **Simularea cu atac fără apărare (Poisoned Simulation)**: Este integrat setul alterat (din etapa 9). M dispozitive capătă rol de clienți malițioși și raportează parametrii compromiși către orchestrator, agregarea făcându-se prin *FedAvg*. Aceasta va măsura impactul loviturii asupra performanței totale.

13. **Simularea sub atac și cu protecție activă (Poisoned + Defense)**: Aceeași simulare cu M clienți atacatori este reluată folosind de data această metodă de agregare robustă ce trebuie să filtreze ponderile malițioase.

14. **Generarea rezultatelor finale (Generate Final Results)**: Orchestratorul cumulează rezultatele din toate simulările (scenariul normal, overheadul apărării, capacitatea de distrugere a atacului și reușita apărării) și generează rapoarte structurate, oferite utilizatorului direct pe ecran.

Arhitectura propusă oferă următoarele beneficii:

- **Flexibilitate**: Sistemul de șabloane permite testarea oricărui model TensorFlow sau PyTorch fără modificări ale infrastructurii. Izolarea mediilor Python elimină conflictele dintre versiuni diferite ale bibliotecilor.

- **Scalabilitate**: Alocarea dinamică a GPU-urilor permite rularea simultană a trei simulări, maximizând utilizarea resurselor disponibile. Structura modulară permite extinderea ulterioară cu mai multe GPU-uri sau chiar cu mai multe servere.
- **Reproducibilitate**: Toate configurațiile și rezultatele sunt stocate în PostgreSQL, permițând replicarea exactă a experimentelor. Mediile conda imutabile garantează consistența dintre execuții.
- **Transparență**: Monitorizarea în timp real prin WebSocket oferă vizibilitate completă asupra procesului de execuție, facilitând depanarea eventualelor probleme.
- **Comparabilitate**: Funcționalitatea de comparare a simulărilor permite evaluarea impactului diferitelor tipuri de atacuri asupra aceluiași model, datele fiind centralizate într-o structură unificată pentru analize statistice avansate.
- **Securitate**: Prevenirea execuției de cod malițios se realizează printr-un proces strict de analiză statică a codului, încă din etapa pre-orchestrării. Atunci când utilizatorul încarcă șablonul Python, se rulează utilitarul avansat de analiză de securitate `semgrep` alături de scriptul personalizat `verify_template.py`. Aceste elemente identifică vulnerabilitățile codului propriu-zis de manipulare și transfer, descoperă injecțiile (*code injection*) și validează prezența obligatorie a structurilor critice, blocând execuțiile periculoase chiar de la nivelul backend-ului.

Imaginea 3.2 prezintă întregul flux al arhitecturii, precum și detaliile tehnice utilizate.

3.2.1 Scenariul complet de utilizare al platformei

Din perspectiva utilizatorului final, platforma urmează un scenariu de utilizare liniar, construit astfel încât să reducă efortul de configurare și să permită repetarea ușoară a experimentelor. În primul rând, utilizatorul se autentifică în aplicație și își creează un proiect nou, care devine spațiul logic în care vor fi organizate șabloanele de antrenare și rezultatele asociate.

În etapa următoare, utilizatorul încarcă sau editează un șablon Python ce descrie modelul, funcțiile de prelucrare a datelor și logica de antrenare locală. Platforma validează structura minimă a acestui șablon și verifică existența funcțiilor obligatorii, astfel încât erorile de bază să fie detectate înaintea consumării resurselor de calcul.

După definirea șablonului, utilizatorul configurează parametrii simulării: numărul de clienți federați, numărul de runde, numărul de epoci locale, tipul atacului, intensitatea otrăvirii, procentajul de date compromise și mecanismul de agregare utilizat. Dacă este necesar, acesta poate selecta și funcții personalizate de agregare sau de poisoning, deja salvate anterior în platformă.

Odată lansată simularea, cererea este transmisă backend-ului, apoi orchestratorului de pe serverul de simulare. În acest punct, platforma alocă resursele disponibile, pregătește directorul de lucru, descarcă datele și modelul necesar, aplică atacul configurat și execută succesiv scenariile curate și atacate. Utilizatorul poate urmări progresul în interfață prin mesaje de stare și poate anula simularea dacă observă o configurație incorectă sau dacă dorește eliberarea resurselor.

La final, rezultatele sunt prezentate într-o formă comparativă, care evidențiază diferențele dintre scenariile *Clean*, *Clean + Defense*, *Poisoned* și *Poisoned + Defense*. Utilizatorul poate analiza metricile obținute, poate compara două simulări diferite și poate relua ulterior experimentul pornind de la aceeași configurație. În acest fel, platforma nu funcționează doar ca un executant de scripturi, ci ca un mediu complet de lucru pentru experimente reproductibile în domeniul securității învățării federate.

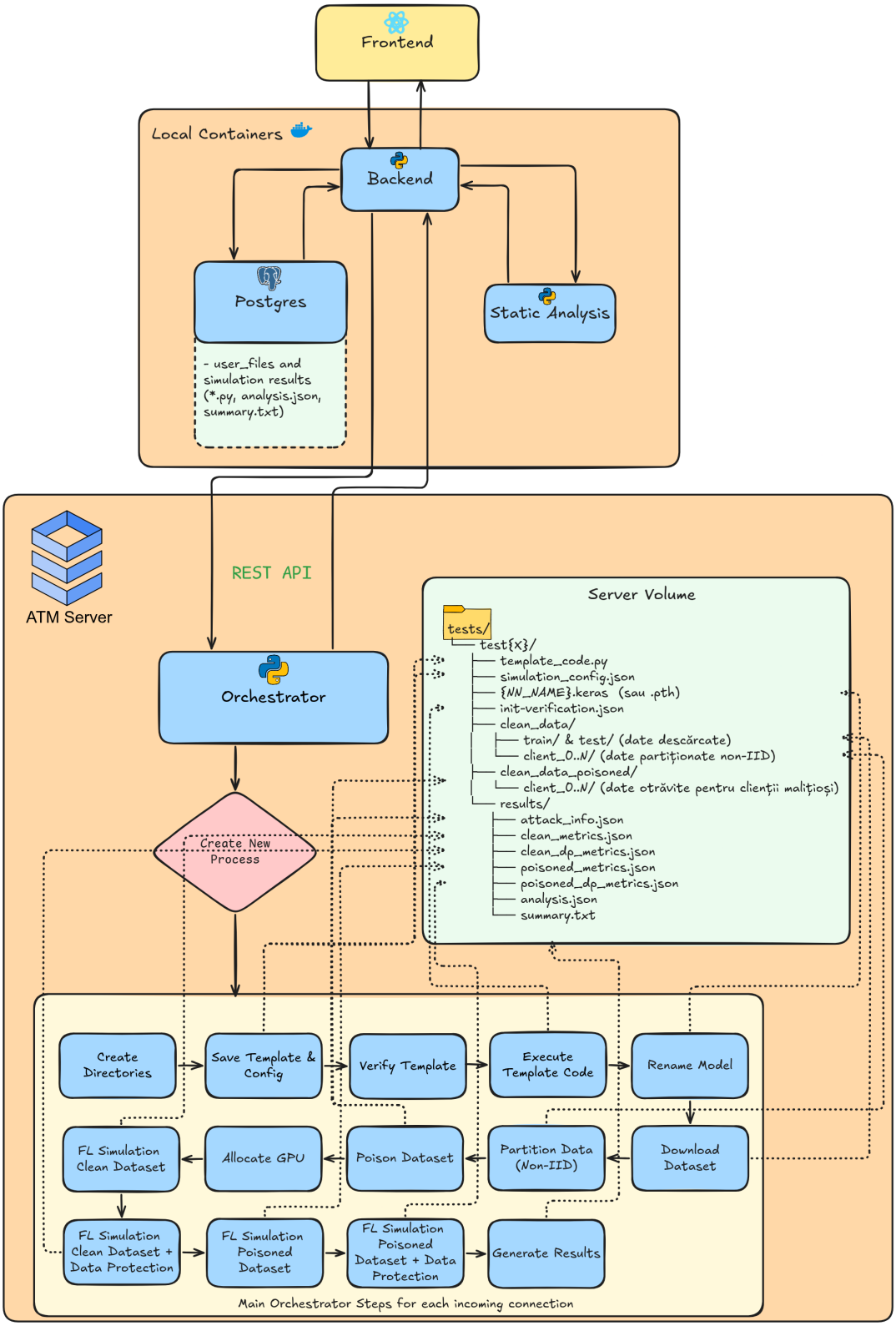

Figura 3.2: Arhitectura și fluxul platformei

3.2.2 Structura directoarelor și a fișierelor importante

Din punct de vedere software, platforma este organizată modular, astfel încât fiecare componentă majoră să poată fi întreținută și extinsă independent. La nivelul serverului local există

directoare dedicate frontend-ului, backend-ului, bazei de date și fișierelor de orchestrare Docker, iar pe serverul de simulare se găsesc scripturile care coordonează antrenarea, alocarea resurselor și procesarea atacurilor.

Componenta frontend include fișierele interfeței React, componentele vizuale, editorul de cod și logica de configurare a proiectelor și simulărilor. Backend-ul local conține endpoint-urile FastAPI, modelele de date, logica de autentificare, persistența configurațiilor și comunicația cu orchestratorul remote. Această separare clară permite tratarea distinctă a preocupărilor de interfață, logică de business și execuție experimentală.

La nivelul serverului de simulare, câteva fișiere au rol central în fluxul de lucru:

- `orchestrator_gpu.py` coordonează întregul pipeline de simulare;
- `gpu_manager.py` gestionează distribuirea resurselor GPU;
- `verify_template.py` validează șablonul încărcat;
- `train_model.py` verifică antrenabilitatea modelului;
- `poison_data.py` aplică atacul de otrăvire;
- `fd_simulator.py` execută rundele federate și produce rezultatele finale.

Pe lângă aceste fișiere principale, fiecare simulare dispune de un director de lucru propriu, creat dinamic pentru utilizatorul și task-ul curent. În acest director sunt stocate temporar șablonul Python executat, datele curate și otrăvite, modelele intermediare și rapoartele rezultate. Organizarea per-task simplifică izolarea execuțiilor, depanarea și refacerea experimentală a unui scenariu anterior.

O asemenea structură a directoarelor nu este importantă doar din punct de vedere practic, ci și metodologic. Ea face posibilă trasabilitatea completă a fiecărei rulări și permite extinderea platformei prin adăugarea de noi module, fără a afecta componentele deja stabile.

3.2.3 Analiza statică de securitate cu Semgrep

Deoarece platforma permite utilizatorilor să încarce cod Python arbitrar care va fi executat pe serverul de simulare, securitatea codului primit este critică. Pipeline-ul de analiză statică utilizează utilitarul *Semgrep* (dezvoltat inițial sub marca *OpenGrep*), un instrument de analiză statică, open-source, bazat pe potrivire de tipare (*pattern matching*) care detectează vulnerabilități de securitate, defecte de cod și anti-șabloane fără a executa codul efectiv.

Procesul de scanare funcționează astfel:

1. Codul primit de la utilizator este scris într-un fișier temporar cu extensia `.py`;
2. Se execută comanda `semgrep scan --config auto --json`, care aplică automat setul complet de reguli comunitare și profesionale;
3. Rezultatul în format JSON este analizat, iar vulnerabilitățile cu severitate ridicată (`HIGH`) sunt extrase;
4. Dacă se detectează cel puțin o vulnerabilitate critică, încărcarea funcției este respinsă, iar utilizatorului i se prezintă detalii despre regulile încălcate (identificatorul regulii, linia de cod afectată și descrierea vulnerabilității);

5. Fișierul temporar este șters imediat după finalizarea scanării.

Pe lângă regulile standard, scanarea exclude explicit reguli specifice ecosistemului *Machine Learning* care generează fals pozitive în contextul simulărilor FL — de exemplu, utilizarea legitimă a funcțiilor `torch.save/load` sau a modulului `pickle`, necesare pentru serializarea și deserializarea modelelor PyTorch și TensorFlow.

În situații excepționale (timeout, absența utilitar *Semgrep* pe server), sistemul adoptă o politică permisivă (*fail-open*) și permite continuarea simulării, în loc să blocheze utilizatorul. Această decizie reflectă natura experimentală a platformei, fiind dublată de izolarea proceselor în medii sandbox.

3.2.4 Funcții personalizate de agregare și otrăvire

Pe lângă algoritmii de agregare robusti și metodele de atac predefinite, platforma oferă utilizatorilor posibilitatea de a defini funcții personalizate, atât pentru procesul de agregare a ponderilor, cât și pentru cel de otrăvire a datelor. Această funcționalitate extinde semnificativ capacitatea de experimentare, permițând cercetătorilor și studenților să testeze ipoteze proprii fără a modifica codul sursă al simulatorului.

3.2.4.1 Funcții personalizate de agregare

Utilizatorul poate defini o funcție de agregare proprie prin intermediul unui editor de cod integrat accesibil din interfața web. Codul trebuie să conțină obligatoriu o funcție cu semnătura `custom_aggregate`, care primește drept parametri listele de ponderi ale clienților, dimensiunile seturilor de date respective, ponderile curente ale modelului global și numărul de clienți malițioși, iar în urma apelului returnează lista de ponderi agregate cu aceeași structură ca intrarea. De menționat aici că platforma nu oferă posibilitatea de a verifica rezultatul de ieșire al funcției, ci doar dacă aceasta respectă semnătura și dacă nu conține vulnerabilități de securitate. Astfel, utilizatorul este responsabil pentru corectitudinea implementării funcției sale. La încărcarea codului, platforma execută un pipeline de validare în trei etape:

1. **Validare sintactică**: codul este analizat prin modulul Python `ast.parse()` pentru a verifica corectitudinea sintaxei;

2. **Verificare structurală**: se parcurge arborele sintactic abstract (AST) pentru a confirma existența funcției `custom_aggregate`;

3. **Analiză statică de securitate**: codul este scanat cu utilitarul *Semgrep* (OpenGrep) pentru identificarea vulnerabilităților de severitate ridicate.

Dacă toate verificările sunt trecute cu succes, funcția este transmisă serverului orchestrator și salvată ca modul Python independent în directorul utilizatorului. În timpul simulării, funcția este încărcată dinamic și apelată de procesul de agregare în locul algoritmului standard. În situația în care execuția funcției personalizate eșuează, sistemul revine automat la algoritmul *FedAvg*, asigurând continuitatea simulării. Funcția apare în interfață sub forma `@nume_functie`, ca o convenție față de algoritmii predefiniți.

3.2.4.2 Funcții personalizate de otrăvire a datelor

Similar funcțiilor de agregare, utilizatorul poate defini o funcție proprie de otrăvire a datelor, cu semnătura `custom_poison`. Aceasta primește imaginea originală, lista claselor disponibile, clasa curentă a imaginii, clasa țintă, precum și parametrii de intensitate și procentaj configurați din interfață. Funcția returnează un tuplu format din imaginea modificată și noua clasă atribuită.

Procesul de validare este identic cu cel al funcțiilor de agregare (validare sintactică, verificare structurală a funcției `custom_poison` și analiza de securitate *Semgrep*). La execuția simulării, dacă operația de otrăvire selectată începe cu prefixul `@`, scriptul `poison_data.py` încarcă dinamic modulul Python corespunzător și aplică funcția personalizată asupra fiecărei imagini din subsetul selectat pentru otrăvire. Dacă funcția generează o eroare la execuție, simularea este întreruptă automat, deoarece datele compromise inconsistente ar invalida rezultatele experimentului.

3.2.4.3 Avantaje și limitări ale extensibilității prin funcții custom

Introducerea funcțiilor personalizate reprezintă unul dintre avantajele importante ale platformei, deoarece transformă simulatorul dintr-un instrument cu opțiuni fixe într-un cadru extensibil de cercetare. Utilizatorul poate defini rapid o nouă funcție de agregare sau o nouă metodă de otrăvire fără a modifica nucleul aplicației și fără a reconstrui infrastructură completă. Această flexibilitate reduce timpul necesar prototipării unei idei și facilitează compararea ei cu mecanismele deja integrate.

Un alt avantaj constă în reproductibilitatea controlată. Odată salvate pe server, funcțiile personalizate pot fi reutilizate în mai multe simulări, ceea ce permite repetarea experimentelor în condiții similare și compararea rezultatelor pe seturi diferite de parametri. Din această perspectivă, funcțiile custom contribuie direct la valoarea platformei ca instrument de cercetare.

Totuși, extensibilitatea prin funcții custom introduce și limitări. În primul rând, validarea automată poate confirma sintaxa, semnătura și absența unor vulnerabilități evidente, dar nu poate garanta corectitudinea științifică a algoritmului implementat. O funcție poate fi formal validă și totuși să producă rezultate greșite sau instabile.

În al doilea rând, aceste funcții pot avea un cost computațional imprevizibil. O metodă de agregare foarte complexă sau o tehnică de poisoning costîșitoare poate influența timpul total de execuție și poate face mai dificilă compararea directă cu metodele predefinite. De aceea, flexibilitatea oferită de platformă trebuie însoțită de o disciplină experimentală atentă.

În al treilea rând, funcțiile custom sunt constrânse de contractul de interfață definit de platformă. Această restricție este necesară pentru siguranță și compatibilitate, însă înseamnă și că nu orice algoritm din literatura de specialitate poate fi integrat direct fără adaptări suplimentare.

Prin urmare, extensibilitatea prin funcții personalizate oferă un echilibru între libertatea de experimentare și controlul infrastructural. Ea extinde capacitatea platformei de a testa ipoteze noi, dar nu elimină responsabilitatea cercetătorului de a valida metodologic rezultatele obținute.

3.2.5 Persistența datelor și structura rezultatelor generate

Pentru a permite reluarea analizelor și compararea simulărilor în timp, platforma implementează un mecanism de persistență împărțit între baza de date PostgreSQL și sistemul de fișiere al serverului de simulare. În baza de date sunt stocate informațiile descriptive despre utilizatori, proiecte, șabloane, parametrii experimentali și starea simulărilor, în timp ce pe disc sunt păstrate artefactele efective rezultate din execuție.

Această separare are un rol funcțional clar. Metadatele din baza de date pot fi interogate rapid pentru listarea simulărilor, filtrarea lor după parametri și compararea mai multor rulări. În schimb, fișierele generate în timpul simulării, cum ar fi modelele, datele intermediare și rapoartele, sunt păstrate separat pentru a nu încărca inutil baza de date cu conținut voluminos.

Fiecare task de simulare primește un director de lucru propriu, în care se salvează șablonul Python executat, versiunea descărcată sau generată a modelului, datele curate și otrăvite, precum și fișierele intermediare produse în timpul antrenării. Această organizare permite reconstituirea ulterioară a unei rulări și simplifică procesul de audit și depanare.

Rezultatele finale sunt structurate în jurul unor fișiere standardizate. Fișierul `analysis.json` conține metricile numerice într-un format ușor de prelucrat automat, `summary.txt` oferă un rezumat textual al simulării, iar `attack_info.json` reține informațiile despre configurația atacului folosit. Împreună, aceste fișiere descriu atât performanța modelului în cele patru scenarii de rulare, cât și contextul experimental care a produs rezultatele.

Pe lângă fișierele finale, sistemul poate genera și fișiere intermediare de verificare și loguri de progres. Deși ele nu sunt destinate neapărat analizei finale de către utilizator, aceste artefacte au valoare în depanare, în verificarea consistenței fluxului de simulare și în justificărea unor diferențe între două rulări aparent similare.

Prin combinarea persistenței relaționale cu stocarea artefactelor pe disc, platforma oferă atât trasabilitate administrativă, cât și trasabilitate tehnică. Această abordare este importantă într-un context experimental, deoarece permite nu doar vizualizarea rapidă a rezultatelor, ci și reconstrucția completă a condițiilor în care acestea au fost obținute.

3.3 Cerintele Software

Cerințele software ale platformei au fost documentate conform standardului IEEE 830-1998 pentru Specificații ale Cerințelor Software (SRS). Fiecare cerință este identificată printr-un cod unic: CF-XX pentru cerințe funcționale, CNF-XX pentru cerințe nefuncționale și CS-XX pentru cerințe de securitate. Cerințele sunt clasificate după gradul de obligativitate: **OBLIGATORIU** (Faza 1), **RECOMANDAT** (Faza 2) și **OPȚIONAL** (Faza 3).

3.3.1 Cerintele functionale

Cerințele funcționale definesc comportamentul specific al platformei și sunt organizate pe module de funcționalitate. În continuare sunt prezentate principalele categorii.

3.3.1.1 Gestionarea și Autentificarea Utilizatorilor (CF-01 – CF-15)

Platforma implementează un sistem complet de management al utilizatorilor:

- Proces de înregistrare cu colectare de informații pe două niveluri: date obligatorii (nume, prenume, email, parolă) și date opționale (organizație, domeniu cercetare)
- Autentificare bazată pe email și parolă, cu parole stocate în format hash folosind algoritmul bcrypt
- Generare și gestionare de token-uri JWT (JSON Web Tokens) pentru sesiuni, cu expirare după 24 de ore de inactivitate
- Autorizare bazată pe roluri (RBAC) cu două roluri: utilizator standard și administrator

- Validare formatul email-ului și complexitatea parolei la înregistrare (minimum 8 caractere, litere mari/mici, cifre)
- Funcționalitate de logout care invalidează token-ul JWT al utilizatorului
- Interfață în care utilizatorul își alege parametrii de simulare ai atacului și ai infrastructurii federate

3.3.1.2 Parametrii de Simulare și Vectori de Atac (CF-16 – CF-25)

Sistemul permite configurarea completă a mediului de simulare:

- Numărul total de dispozitive/clienți participanți (range: 2–100)
- Numărul de dispozitive compromise (range: 1 până la 50% din total)
- Numărul total de runde de antrenare (range: 1–500) și numărul de runde corupte per dispozitiv
- Strategia de distribuire a dispozitivelor corupte (primele, ultimele, distribuție uniformă)
- Suport pentru 7 tipuri de atacuri predefinite (Label Flip, Backdoor BadNets, Backdoor Blended, Backdoor Sinusoidal, Backdoor Trojan, Backdoor Semantic și Edge-Case Backdoor), precum și posibilitatea definirii de funcții personalizate de otrăvire
- Configurarea intensității atacului specific fiecărui tip (deviație standard, procentaj etichete, dimensiune trigger)
- Selecția funcției de agregare din algoritmii integrați (FedAvg, Krum, Trimmed Mean, Median, FoolsGold, Norm Clipping, Random) sau din funcții de agregare personalizate definite de utilizator
- Validarea valorilor parametrilor introduși cu mesaje de eroare pentru valori invalide

3.3.1.3 Funcții Personalizate și Securitate Cod (CF-73 – CF-82)

- Definirea de funcții personalizate de agregare prin editor de cod Python integrat (Monaco Editor)
- Definirea de funcții personalizate de otrăvire a datelor prin editor de cod Python dedicat
- Validare sintactică automată a codului Python folosind modulul `ast.parse()`
- Verificare structurală a prezenței funcțiilor obligatorii (`custom_aggregate` sau `custom_poison`)
- Analiza statică de securitate a codului cu utilitarul Semgrep (OpenGrep) pentru detecția vulnerabilităților de tip RCE (Remote Code Execution)
- Respingerea automată a codului care conține vulnerabilități de severitate ridicată, cu prezentarea detaliată a regulilor încălcate
- Persistarea funcțiilor personalizate pe serverul orchestrator și listarea lor la reconectare
- Suport pentru ștergerea funcțiilor personalizate cu actualizare în timp real a listelor din interfață

- Încărcare dinamică a modulelor Python în timpul simulării prin `importlib`
- Mecanism de fallback automat la FedAvg în cazul eșecului funcțiilor de agregare personalizate

3.3.1.4 Interfețe de Comunicare (CF-26 – CF-29)

- Rețea de containere Docker interconectate pentru comunicarea frontend-backend
- REST API pentru comunicarea dintre serverul central și serverul de simulare
- Monitorizarea procesului de simulare prin mesaje de logging folosind WebSockets
- Mecanism de load-balancing prin crearea de task-uri asincrone pentru simulări multiple

3.3.1.5 Integrarea Tehnologiilor (CF-30 – CF-33)

- Editor de text performant (Monaco Editor) pentru editarea scripturilor Python
- Biblioteci de vizualizare a informației (Bar Chart) pentru analiza rezultatelor
- Suport pentru simularea cu modele TensorFlow și PyTorch
- Suport pentru încărcare de modele preantrenate din repository-uri publice (HuggingFace)

3.3.1.6 Interpretarea Rezultatelor (CF-34 – CF-38)

- Afișarea rezultatelor simulării cu metrici detaliate (acurațete, precizie, recall, F1-score) per scenariu
- Compararea side-by-side a două simulări, inclusiv configurația FL, parametrii atacului și toate metricile
- Vizualizare grafică interactivă cu bar charts grupate pentru mai multe simulări
- Export rezultate individuale în format PDF (configurație, metrici, matrice de confuzie, rezumat)
- Export multiplu în format CSV cu toate metricile per scenariu în format tabular

3.3.1.7 Gestionarea Proiectelor și Fișierelor (CF-39 – CF-48)

- Creare și organizare ierarhică a proiectelor de simulare
- Creare, ștergere, redenumire și reatribuire a fișierelor de simulare între proiecte
- Stocare proiecte și fișiere în baza de date PostgreSQL cu izolarea datelor per utilizator
- Reordonare fișiere prin drag-and-drop

3.3.1.8 Executarea și Monitorizarea Simulărilor (CF-49 – CF-56)

- Transmiterea scriptului de simulare și parametrilor către server prin REST API
- Execuție pe server dedicat cu resurse GPU alocate dinamic
- Afișarea progresului în timp real (rundă curentă, GPU alocat, status) prin WebSocket
- Anulare simulări în curs de execuție cu eliberarea automată a resurselor
- Gestionarea erorilor cu mesaje descriptive transmise utilizatorului
- Stocare rezultate finalizate în baza de date pentru consultare ulterioară

3.3.1.9 Gestionarea Modelelor (CF-57 – CF-62)

- Furnizare șablon de cod Python predefinit la crearea unui fișier de simulare, cu funcții necesare antrenării
- Încărcare modele preantrenate din HuggingFace cu descărcare automată în timpul simulării
- Editare și personalizare a funcțiilor din șablon pentru adaptarea la specificul modelului
- Validare sintaxa codului Python înainte de lansarea simulării și semnalarea erorilor
- Documentație inline și comentarii explicative în șablon pentru ghidarea utilizatorului

3.3.1.10 Procesarea și Orchestrarea pe Server (CF-63 – CF-72)

- Recepționare asincronă a cererilor de simulare prin REST API
- Creare proces dedicat pentru fiecare cerere cu toate informațiile necesare
- Descărcare model/rețea preantrenată din repository-ul specificat
- Încărcare model în memoria GPU alocat specific task-ului
- Descărcare set de date și aplicare atac data poisoning conform parametrilor
- Execuție antrenare în patru scenarii distincte: Clean, Clean+Defense, Poisoned, Poisoned+Defense
- Returnare rezultate antrenare cu metrici complete și monitorizare continuă a proceselor

3.3.2 Cerințele nefuncționale

Cerințele nefuncționale definesc atributele calității și performanței pe care sistemul trebuie să le satisfacă.

3.3.2.1 Cerințe de Performanță (CNF-01 – CNF-17)

- Interfața web va răspunde la acțiunile utilizatorului în maximum 2 secunde pentru operații standard
- API-ul backend va procesa cererile de autentificare în maximum 1 secundă
- Încărcarea dashboard-ului principal va fi completă în maximum 3 secunde
- Actualizările în timp real prin WebSocket vor avea o latență de maximum 500ms
- Sistemul va suportă minimum 50 de utilizatori concurenți fără degradare semnificativă
- Serverul de simulări va putea procesa cel puțin 5 simulări simultane pe un singur GPU
- API-ul va gestiona minimum 100 de cereri pe secundă în condiții normale
- Fiecare simulare va fi limitată la maximum 10 GB memorie GPU
- Containerizarea cu Docker va permite deploy-uri reproductibile în diferite medii

3.3.2.2 Cerințe de Securitate (CS-01 – CS-17)

- Autentificare bazată pe JWT cu expirare configurabilă, semnate cu algoritm HS256
- Parole stocate în format hash folosind algoritmul bcrypt
- Control de acces bazat pe roluri (RBAC) cu roluri user și admin
- Comunicații criptate folosind TLS 1.2 sau superior
- Izolarea datelor între utilizatori (multi-tenancy)
- Respectarea principiilor GDPR privind colectarea și procesarea datelor personale
- Scripturile de simulare încărcate vor fi executate în medii izolate (sandbox)
- Înregistrarea tuturor încercărilor de autentificare și a acțiunilor administrative în jurnalul de audit

3.3.2.3 Fiabilitate și Disponibilitate (CNF-18 – CNF-32)

- Disponibilitate țintă de 99% în orele de lucru (8:00–22:00)
- Implementare graceful degradation — eșecul unei componente nu va afecta întregul sistem
- Conexiunile WebSocket vor avea mecanism de reconectare automată
- Backup-uri ale bazei de date efectuate zilnic
- Erorile vor fi afișate utilizatorilor în mesaje clare, iar detaliile tehnice vor fi înregistrate în log-uri
- Simulările eșuate vor genera notificări cu detalii despre cauza erorii

3.3.2.4 Accesibilitate (CNF-33 – CNF-41)

- Interfață intuitivă ce permite utilizatorilor noi să lanseze o simulare în maximum 10 minute
- Feedback vizual clar pentru toate acțiunile, cu etichete descriptive pentru formulare și butoane
- Interfață responsive funcțională pe rezoluții de minimum 1366x768
- Opțiune de temă clară și întunecată (light/dark mode) cu preferință persistentă
- Navigare consistentă pe toate paginile aplicației

3.3.2.5 Mentenabilitate și Portabilitate (CNF-42 – CNF-51)

- Cod sursă versionat folosind Git, cu convenții de stil consistente (ESLint pentru JavaScript, PEP8 pentru Python)
- Arhitectură modulară ce permite actualizarea individuală a componentelor
- Aplicația rulează în containere Docker pentru portabilitate maximă (exceptând serverul de simulare)
- Configurații specifice mediului gestionate prin variabile de mediu
- Sistem instalabil pe Linux, Windows și macOS (prin Docker)
- Interfață web compatibilă cu Chrome 90+, Firefox 88+, Edge 90+ și Safari 14+
- Suport pentru modele TensorFlow 2.x și PyTorch 2.x

3.4 Testare

Testarea platformei de simulare a atacurilor Data Poisoning acoperă multiple niveluri, de la validarea componentelor individuale până la testarea integrată a întregului flux de simulare. Dată fiind natura distribuită a arhitecturii (frontend containerizat, backend containerizat, orchestrator pe server remote cu GPU), strategia de testare se concentrează pe verificarea corectitudinii fiecărei etape din pipeline-ul de simulare, a comunicării între componente și a rezilienței sistemului la erori.

3.4.1 Testarea de Validare a Șabloanelor

Primul nivel de testare are loc înainte de consumarea oricăror resurse computaționale. Modulul `verify_template.py` realizează o **analiză statică** a scriptului de antrenare furnizat de utilizator, verificând:

- Existența tuturor funcțiilor obligatorii în șablon (funcția de descărcare date `download_data()`, funcția de creare model, funcțiile de setare și extragere a ponderilor)
- Conformitatea cu șablonul de interfață așteptat de platforma de simulare
- Absența construcțiilor de cod potențial periculoase sau a importurilor nepermise

Această validare previne erorile tardive în pipeline, economîșind timp și resurse GPU. În cazul în care verificarea eșuează, utilizatorul primește un mesaj descriptiv cu funcțiile lipsă sau structurile incorecte.

3.4.2 Testarea creării de funcții personalizate

Un element important al platformei îl reprezintă posibilitatea extinderii sale prin definirea de funcții personalizate, fără modificarea codului sursă al orchestratorului. Din acest motiv, procesul de testare nu s-a limitat doar la verificarea execuției funcțiilor, ci a urmărit întregul lor ciclu de viață: redactare în editorul web, validare sintactică și structurală, analiză de securitate, persistare pe server și utilizare efectivă într-o simulare. Această categorie de teste confirmă că platforma poate fi extinsă controlat, păstrând în același timp siguranța infrastructurii și coerența rezultatelor experimentale.

3.4.2.1 Testarea funcțiilor personalizate de agregare

Pentru funcțiile de agregare personalizate, testarea a urmărit atât corectitudinea procesului de încărcare, cât și integrarea lor în scenariile federate. Au fost verificate următoarele aspecte:

- validarea sintactică a codului Python introdus în editor, pentru a respinge imediat funcțiile care conțin erori de scriere sau structură
- verificarea existenței funcției obligatorii `custom_aggregate` în arborele sintactic abstract al codului transmis
- rularea analizei de securitate cu *Semgrep*, pentru a bloca încărcarea funcțiilor care includ tipare periculoase
- salvarea corectă a funcției pe serverul orchestrator și listarea ei ulterioară în interfață sub forma `@nume_functie`
- încărcarea dinamică a modulului în timpul simulării și apelarea lui în locul algoritmului standard de agregare
- verificarea mecanismului de fallback la *FedAvg* în cazul în care funcția personalizată produce o eroare la execuție

Prin aceste teste s-a demonstrat că funcțiile de agregare definite de utilizator pot fi utilizate ca mecanisme experimentale de apărare sau ca variante proprii de combinare a ponderilor, fără a compromite continuitatea simulării.

3.4.2.2 Testarea funcțiilor personalizate de otrăvire a datelor

În mod similar, funcțiile personalizate de poisoning au fost testate pentru a confirma că pot modifica seturile de date într-o manieră controlată și reproductibilă. Testarea a inclus următoarele verificări:

- validarea sintactică a codului și confirmarea existenței funcției obligatorii `custom_poison`
- verificarea semnăturii funcției și a compatibilității parametrilor primiți cu fluxul intern al scriptului `poison_data.py`

- analiza statică de securitate a codului înainte de salvare, pentru a preveni încărcarea de funcții malițioase
- aplicarea funcției asupra subsetului de imagini selectat pentru otrăvire și verificarea returnării corecte a perechii imagine modificată – etichetă nouă
- testarea întreruperii controlate a simulării atunci când funcția personalizată produce rezultate invalide sau ridică excepții la execuție
- confirmarea faptului că doar datele marcate pentru compromitere sunt alterate, iar restul setului de antrenare rămâne nemodificat

Aceste teste validează faptul că platforma poate fi folosită și pentru prototiparea rapidă a unor atacuri noi, definite direct de utilizator, menținând trasabilitatea și consistența datelor rezultate.

3.4.3 Scenarii de test negative

Pe lângă scenariile în care platforma funcționează conform fluxului ideal, a fost necesară și testarea explicită a situațiilor de eșec controlat. Aceste scenarii negative confirmă că sistemul nu doar execută corect simulările valide, ci și respinge predictibil intrările sau stările incorecte, fără a compromite resursele și fără a produce rezultate ambigue.

O primă categorie de astfel de scenarii vizează șabloanele invalide. Au fost considerate cazurile în care utilizatorul încarcă un fișier Python cu erori de sintaxă, cu funcții obligatorii lipsă sau cu semnături incompatibile cu interfața cerută de orchestrator. În toate aceste situații, platforma trebuie să blocheze simularea înainte de consumarea resurselor costîșitoare și să furnizeze un mesaj de eroare suficient de clar pentru corectarea rapidă a problemei.

O a două categorie privește funcțiile personalizate. Au fost urmărite cazurile în care funcțiile `custom_aggregate` sau `custom_poison` lipsesc, sunt definite incorect, conțin operații nesigure sau returnează date invalide. Pentru funcțiile de agregare, comportamentul așteptat este revenirea controlată la *FedAvg* atunci când execuția eșuează. Pentru funcțiile de poisoning, sistemul trebuie să întrerupă simularea și să semnaleze explicit cauza, deoarece consistența datelor modificate este critică pentru validitatea rezultatelor.

Un alt set de scenarii negative privește indisponibilitatea resurselor și a conexiunilor. În absența unui GPU disponibil în intervalul de timeout configurat, sistemul trebuie să continue execuția pe CPU. În cazul întreruperii comunicării dintre backend și orchestrator, statusul simulării trebuie tratat robust, fără pierderea informațiilor deja generate.

Au fost incluse și situațiile de anulare sau oprire forțată a unei simulări în etape diferite ale pipeline-ului. În aceste cazuri, arborele de procese trebuie închis corect, GPU-ul eliberat, iar directorul de lucru tratat conform politicilor de curățare ale sistemului. Aceste verificări sunt importante deoarece un sistem care gestionează corect doar scenariile ideale devine rapid instabil în utilizarea reală.

Prin includerea acestor scenarii negative, strategia de testare acoperă nu doar corectitudinea funcțională a platformei, ci și robustețea ei operațională în condiții de eroare.

3.4.4 Testarea Pipeline-ului de Simulare

Pipeline-ul de simulare parcurge 14 etape secvențiale, fiecare cu mecanism propriu de tratare a erorilor. Testarea funcțională verifică:

- **Verificarea corectitudinii modelului (Etapa 4)** — după încărcarea șablonului, se execută o sesiune de antrenare de validare pe GPU-ul alocat. Acuratețea inițială este înregistrată în fișierul `init-verification.json`, servind drept referință pentru evaluări ulterioare
- **Descărcarea și partiționarea datelor (Etapa 6)** — se verifică descărcarea corectă a setului de date prin funcția `download_data()` și partiționarea în N split-uri IID egale (câte unul per client federat), cu setul de test partajat între toți clienții
- **Procesul de otrăvire a datelor (Etapa 9)** — se validează crearea corectă a copiei otrăvite pentru fiecare din cele 7 tipuri de atacuri suportăte, verificând aplicarea parametrilor configurați (intensitate, procentaj, tip trigger)
- **Simulările federate (Etapele 10–13)** — se testează execuția corectă a celor patru scenarii (Clean, Clean+Defense, Poisoned, Poisoned+Defense), verificând că toate pornesc de la aceeași configurație inițială a modelului
- **Generarea rezultatelor (Etapa 14)** — se verifică crearea fișierelor `analysis.json`, `summary.txt` și `attack_info.json` cu metrici complete și corecte

La fiecare etapă, în cazul unei erori, pipeline-ul se oprește, generează un raport de eroare și transmite informația înapoi către utilizator prin WebSocket.

3.4.5 Testarea de Integrare

Testarea de integrare verifică fluxul complet de date între cele trei componente ale arhitecturii:

- **Frontend → Backend**: Trimiterea cererii POST cu șablonul Python și configurația simulării de la interfața React către API-ul FastAPI. Se verifică validarea de securitate a codului, stocarea configurației în PostgreSQL și retransmiterea asincronă către orchestrator
- **Backend → Orchestrator**: Comunicarea REST API între backend-ul containerizat și orchestratorul Flask de pe serverul ATM. Se testează autentificarea cu credențiale, transmiterea parametrilor și primirea status-urilor de progres
- **Orchestrator → Backend → Frontend**: Transmiterea rezultatelor finale de la orchestrator prin polling HTTP, colectarea fișierelor JSON cu metrici și afișarea lor în interfața utilizatorului

3.4.6 Testarea Comunicării WebSocket

Comunicarea în timp real este testată prin:

- Verificarea stabilirii conexiunii WebSocket între frontend și backend
- Testarea loop-ului asincron de polling (la fiecare 2 secunde) al stării simulării de la orchestrator
- Validarea transmiterii corecte a actualizărilor de progres (etapă curentă, status GPU, metrici intermediare
- Testarea mecanismului de reconectare automată în cazul întreruperii conexiunii

3.4.7 Testarea Managementului GPU

Componenta `GPUManager` este testată pentru:

- Interogarea corectă a stării GPU-urilor disponibile prin `nvidia-smi`
- Alocarea thread-safe a GPU-urilor din coada de resurse disponibile
- Respectarea timeout-ului de 10 minute și fallback-ul pe CPU
- Eliberarea garantată a GPU-ului (`finally` block) la finalizarea, anularea sau eșecul simulării
- Distribuirea echilibrată a simulărilor pe cele 3 GPU-uri NVIDIA A40 disponibile

3.4.8 Testarea Fluxului de Anulare

Mecanismul de anulare a simulărilor este testat prin:

- Verificarea interogării stării de anulare la fiecare etapă din pipeline
- Testarea terminării corecte a arborelui de procese (`kill_process_tree()`)
- Validarea eliberării GPU-ului alocat și ștergerii directorului de lucru
- Confirmarea setării corecte a statusului la *cancelled* și notificarea utilizatorului

3.4.9 Testarea de Securitate

Aspectele de securitate testate includ:

- Validarea token-urilor JWT: verificarea semnăturii digitale, expirarea și existența utilizatorului în baza de date pentru fiecare cerere protejată
- Validarea codului Python încărcat de utilizatori: detectarea constructelor periculoase și execuția în medii izolate (medii Anaconda separate)
- Izolarea datelor între utilizatori: verificarea că fiecare utilizator accesează exclusiv propriile proiecte, fișiere și rezultate
- Stocarea securizată a credențialelor prin Docker Secrets (parolă bază de date, cheie JWT, parolă orchestrator)
- Configurarea CORS pentru restricționarea accesului API doar la originile autorizate

3.5 Modele de Analiză

În această secțiune sunt prezentate modelele de analiză ale platformei, incluzând diagramele de context, componente, flux, entitate-relație și cazuri de utilizare. Aceste diagrame au fost elaborate în cadrul documentului de Specificații ale Cerințelor Software (SRS) și oferă o perspectivă vizuală asupra arhitecturii și funcționalităților platformei.

3.5.1 Diagrama de Context

Diagrama de context prezintă marginile platformei și entitățile externe care interacționează cu sistemul. La Nivelul 0, sunt identificați actorii principali: Cercetătorii (Data Scientists), Instituțiile și Administratorul, fiecare cu roluri specifice în cadrul platformei.

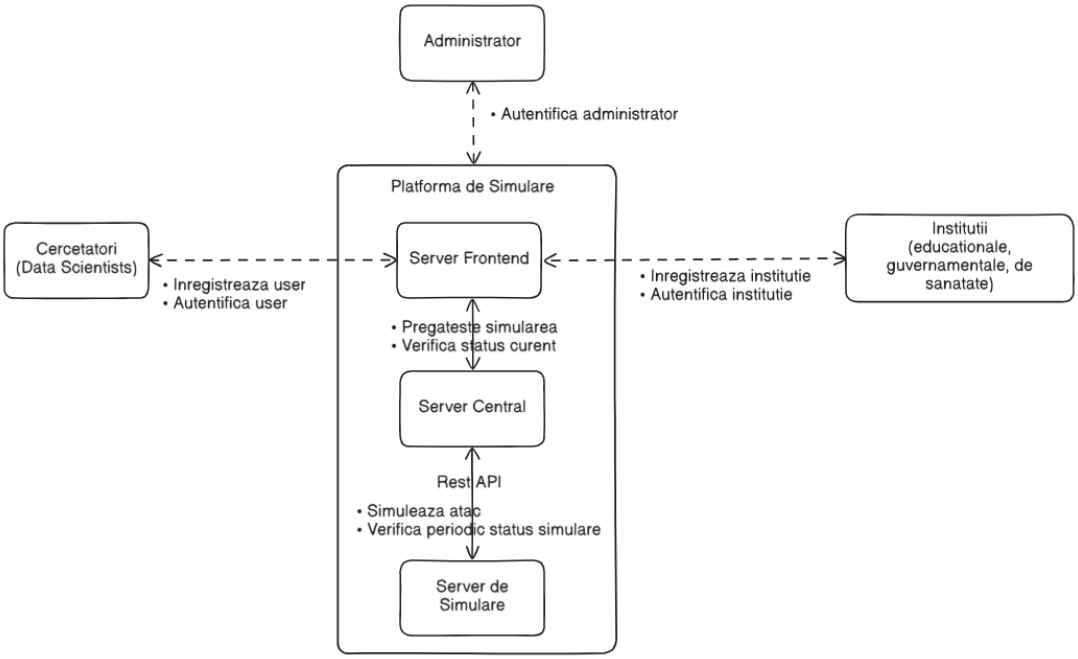

Figura 3.3: Diagrama de Context — Nivel 0: Marginile Platformei

La Nivelul 1, diagrama detaliază fluxul de date între sistem și entitățile externe, evidențiind interacțiunile dintre interfețele publice și protejate ale frontend-ului, serverul backend central cu baza de date PostgreSQL și serverul de simulări.

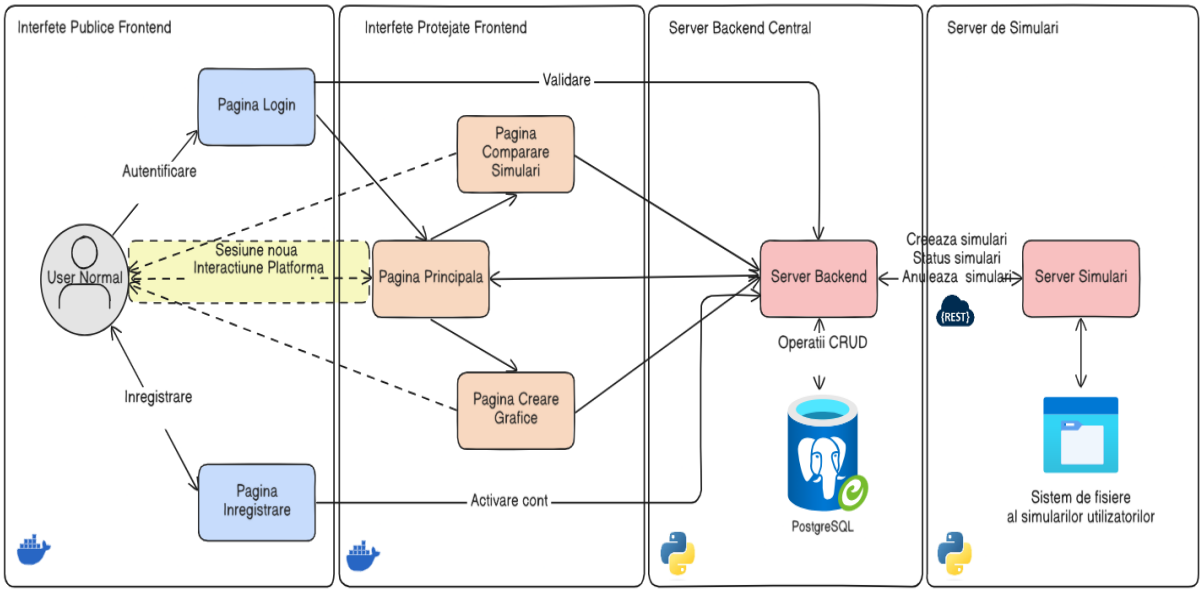

Figura 3.4: Diagrama de Context — Nivel 1: Fluxul de date

3.5.2 Diagrama de Componente

Diagrama de componente prezintă structura internă a platformei, evidențiind cele două servere principale (Server Central și Server de Simulări Remote) împreună cu componentele software asociate: React Frontend, PostgreSQL, Fast API pe serverul central, respectiv GPU Manager și Python Orchestrator pe serverul de simulări. De asemenea, sunt ilustrate fluxurile de date de la configurarea simulării până la generarea rezultatelor finale.

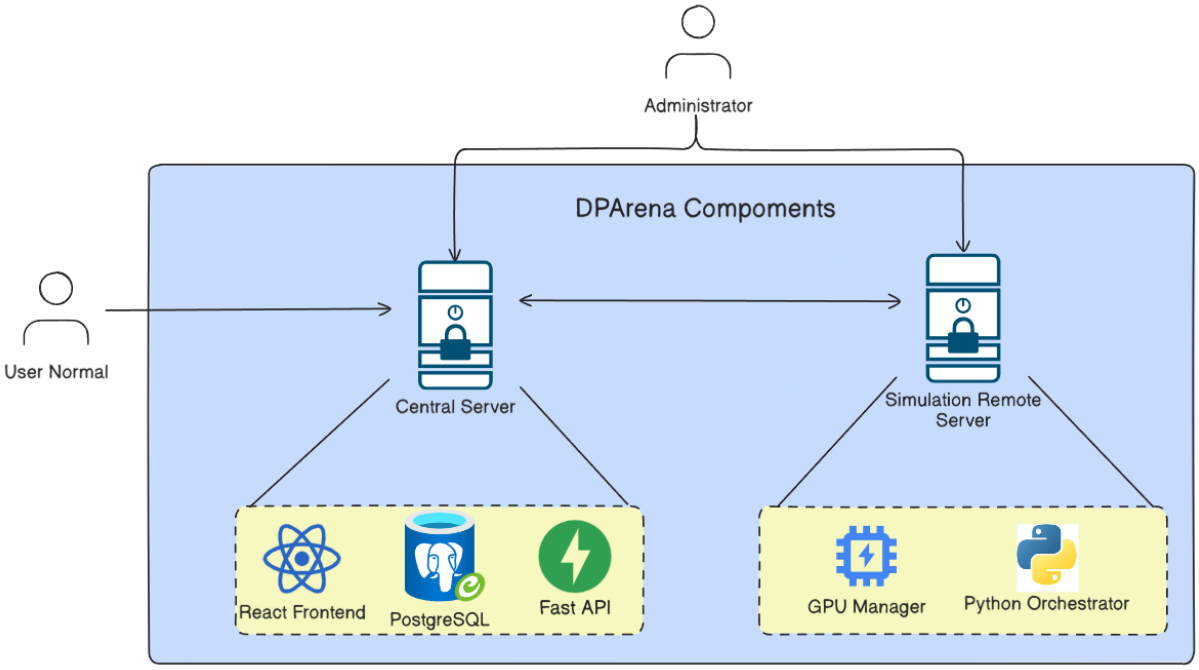

Figura 3.5: Diagrama de Componente a Platformei

3.5.3 Diagramele de Flux

Diagramele de flux prezintă procesele principale ale platformei din perspectiva utilizatorului și a simulării.

Diagrama fluxului utilizatorului ilustrează parcursul complet al unui utilizator în platformă, de la înregistrare și autentificare, prin integrarea multi-nivel (organizație, mediu de cercetare), până la accesul la dashboard și funcționalitățile principale: vizualizarea rezultatelor, crearea de simulări, compararea rezultatelor și gestionarea profilului.

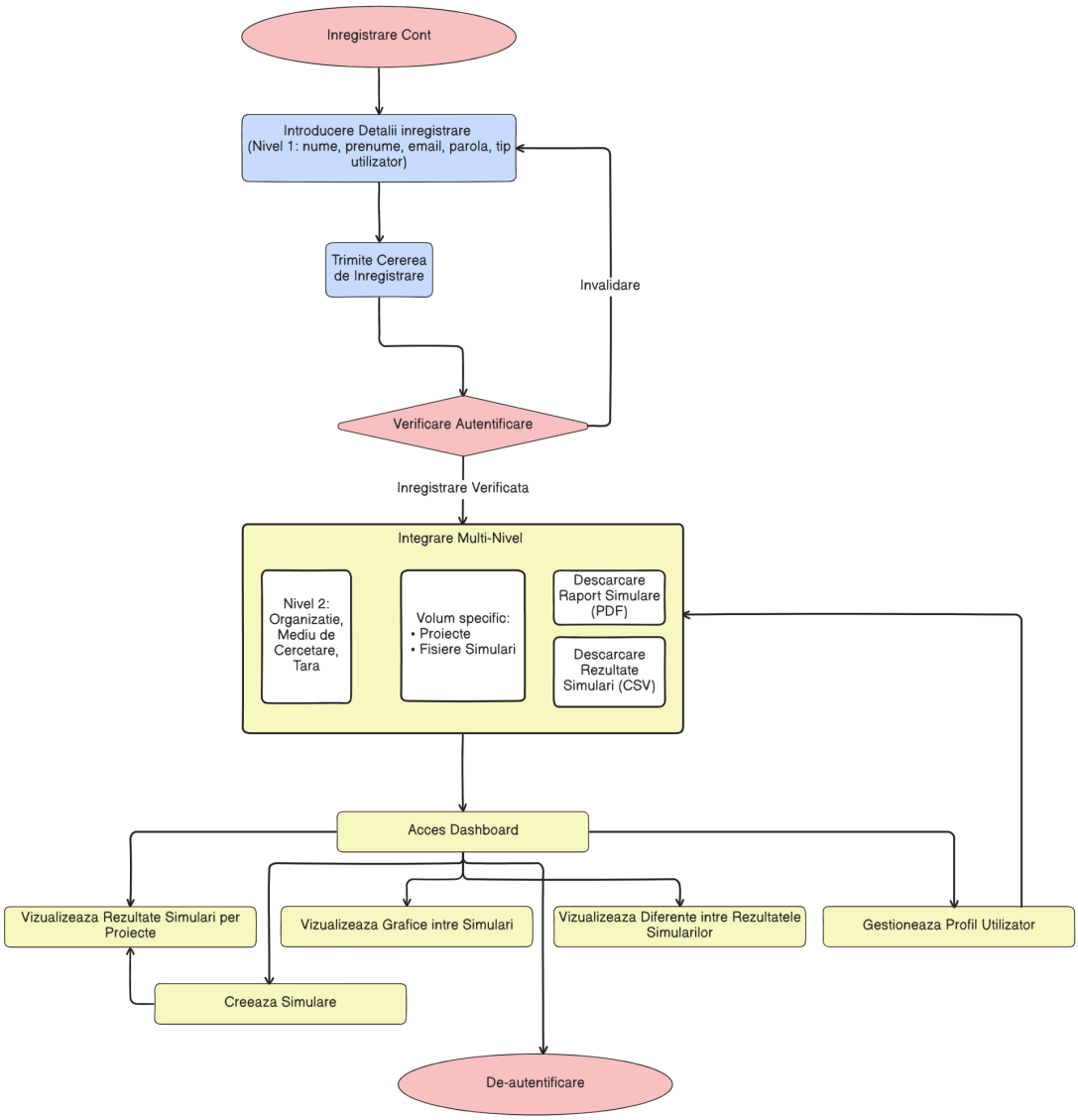

Figura 3.6: Diagrama Flux Utilizator

Diagrama fluxului de simulare detaliază etapele pipeline-ului de simulare, de la alocarea GPU și verificarea template-ului, prin descărcarea rețelei și a setului de date, aplicarea atacului de otrăvire, până la execuția celor trei scenarii de simulare federată (date curate, date otrăvite, date otrăvite cu funcție de agregare) și generarea rezultatelor finale.

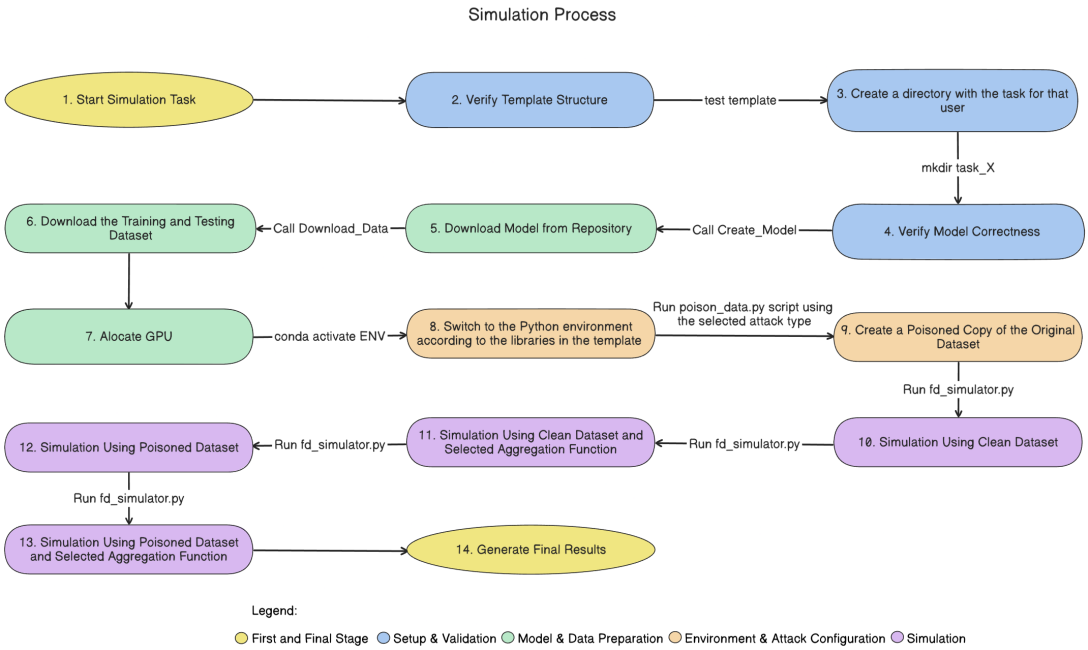

Figura 3.7: Diagrama Fluxului de Simulare

3.5.4 Diagrama Entitate-Relație (ERD)

Diagrama ERD prezintă structura bazei de date PostgreSQL a platformei, cu cele patru tabele principale: `users`, `projects`, `files` și `simulation_results`. Sunt evidențiate relațiile dintre entități: un utilizator creează mai multe proiecte, un proiect conține mai multe fișiere, iar rezultatele simulărilor sunt asociate cu utilizatorul, proiectul și fișierul corespunzător.

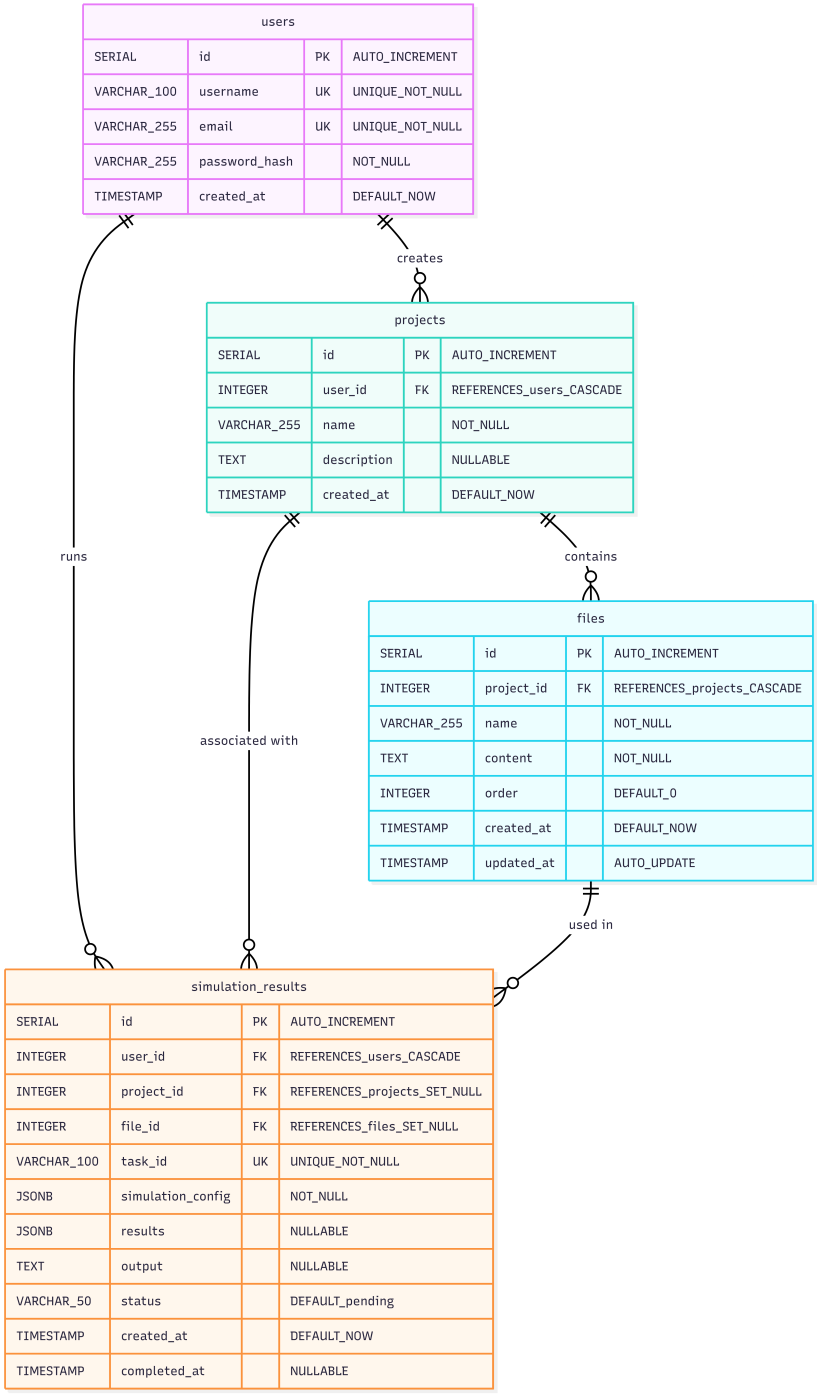

Figura 3.8: Diagrama Entitate-Relație (ERD)

3.5.5 Diagrama Cazurilor de Utilizare

Diagrama cazurilor de utilizare prezintă funcționalitățile platformei din perspectiva celor doi actori principali: Administratorul și Utilizatorul Normal. Administratorul dispune de capabilități de gestionare a conturilor, vizualizare alerte și jurnale de audit, iar utilizatorul normal accesează funcționalitățile de simulare, configurare, monitorizare și analiză a rezultatelor.

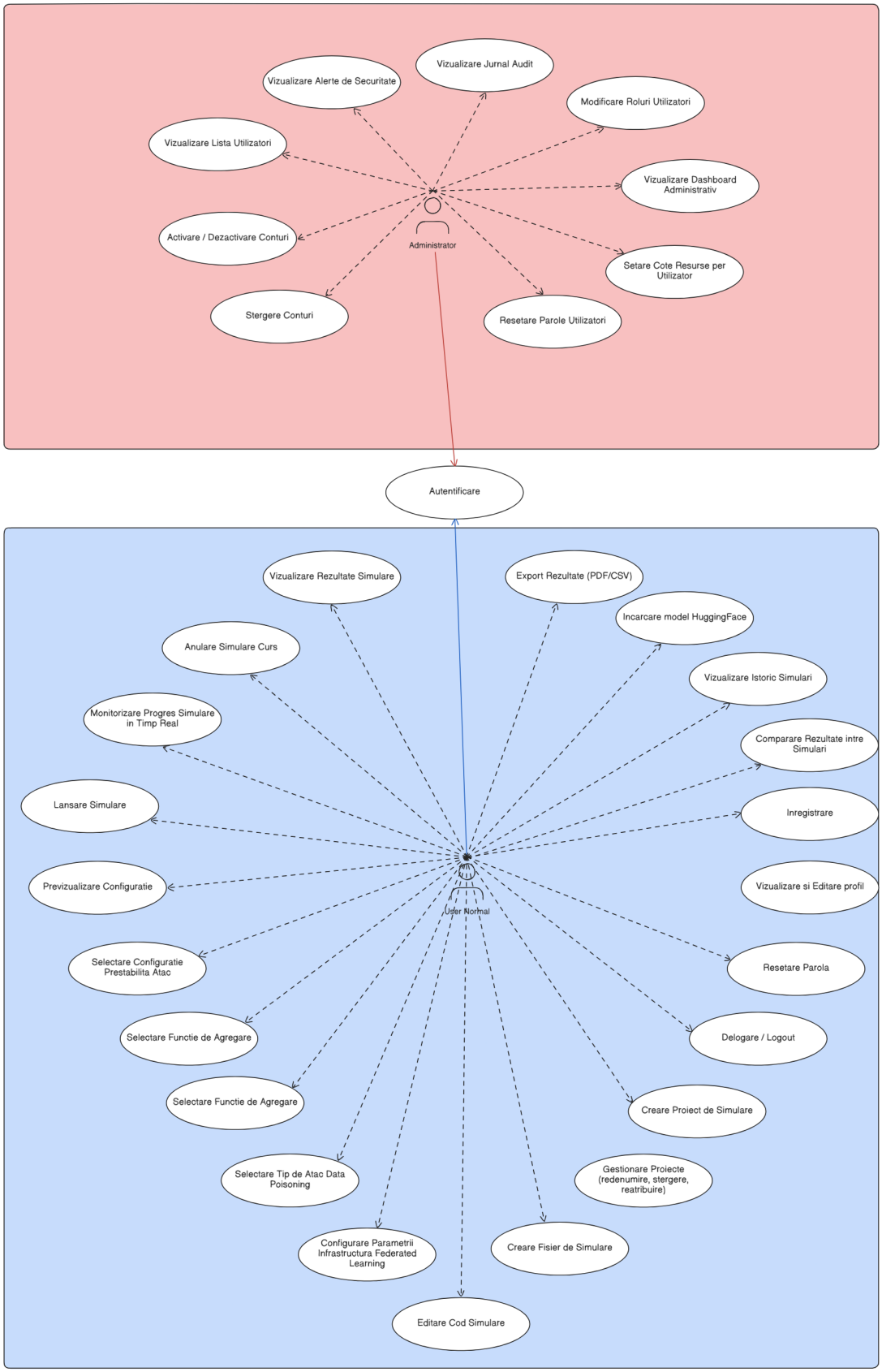

Figura 3.9: Diagrama Cazurilor de Utilizare

Capitolul 4:

Rezultate și Metrici Simulări

Acest capitol prezintă validarea experimentală a platformei. Sunt ilustrate două studii de caz principale care demonstrează modul în care sistemul poate fi utilizat pentru evaluarea mecanismelor de apărare împotriva atacurilor de tip Data Poisoning într-un mediu de învățare federată.

4.1 Metodologia experimentală

Evaluarea platformei a fost realizată printr-o metodologie experimentală orientată spre comparație controlată între scenarii curate și scenarii compromise. În toate experimentele s-a urmărit păstrarea unui număr cât mai redus de variabile modificate simultan, astfel încât efectul atacului sau al mecanismului de apărare să poată fi observat și interpretat mai clar. Din această perspectivă, fiecare studiu de caz a fost construit în jurul unei întrebări experimentale precise: cum se modifică performanța modelului atunci când crește intensitatea otrăvirii și, respectiv, care mecanism de apărare oferă cea mai bună rezistență într-un scenariu de atac fix.

Configurațiile au fost alese astfel încât să fie suficient de complexe pentru a reproduce dificultăți reale, dar și suficient de stabile pentru a permite comparații repetabile. Alegerea setului de date CIFAR-10 și a arhitecturii ResNet18 oferă un echilibru util între relevanța practică și costul computațional. Numărul de clienți, rundele federate și distribuția Non-IID au fost păstrate în intervale care permit observarea efectelor agregării și ale comportamentului malițios fără a transforma experimentul într-un test exclusiv de scalabilitate hardware.

În plus, metodologia utilizată valorifică structura nativă a platformei, care execută pentru aceeași configurație mai multe scenarii: *Clean*, *Clean + Defense*, *Poisoned* și *Poisoned + Defense*. Această organizare permite separarea mai clară a trei dimensiuni esențiale: performanța de referință a modelului, costul introdus de apărare în absența atacului și capacitatea apărării de a limita degradarea sub atac. Astfel, capitolul nu urmărește doar obținerea unor valori numerice, ci și justificărea lor într-un cadru experimental coerent.

4.2 Studiu de Caz 1: Creșterea intensității otrăvirii datelor

Scopul primului experiment este de a evalua modul în care performanța modelului global se degradează pe măsură ce proporția datelor compromise din rețea crește.

4.2.1 Detalii practice

Pentru mediul de antrenare a fost utilizat setul de date CIFAR-10, cuprinzând 60.000 de imagini distribuite în 10 clase. Antrenarea a fost realizată folosind framework-ul PyTorch pe o arhitectură ResNet18 pre-antrenată. Setul de date a fost eșantionat Non-IID (date distribuite neidentic și independent) în mod egal pe toți clienții, fiecare având 80% imagini dintr-o clasă dominantă și 20% din restul claselor. Fiecare rundă de antrenare locală a utilizat optimizatorul Adam cu o rată de învățare mică (0.001) și funcția de pierdere de tip Categorical Cross-Entropy.

Simularea a fost executată în șapte instanțe distincte, incrementând progresiv proporția datelor compromise din setul total: 1%, 5%, 10%, 15% și până la 55%. Pentru nodurile compro-

mise, a fost aplicată tehnica de otrăvire „Label-Flipping” (inversarea etichetelor). Pentru a apăra rețeaua împotriva acestui atac, orchestratorul a folosit metoda robustă de agregare *FoolsGold*, care analizează similaritatea cosinusului actualizărilor pentru a penaliza contribuitorii malițioși. Întregul mediu a fost accelerat de unități GPU pentru a gestiona eficient cerințele riguroase de calcul.

4.2.2 Rezultate experimentale

Efectuarea tuturor etapelor de simulare pentru un număr fix de 10 clienți ($N = 10$), fiecare mediu federat rulând pentru 5 runde a câte 3 epoci pe rundă, a durat aproximativ două ore, un timp puternic influențat de distribuția Non-IID a datelor.

Etapa curată (Clean) și etapa exclusiv atacată (Poisoned) au utilizat algoritmul standard de agregare FedAvg, în timp ce instanțele rulate cu apărare au implementat metoda FoolsGold. Așa cum a fost demonstrat și în literatura de specialitate, cu cât au fost prezenți mai puțini clienți infectați, cu atât eficacitatea protecției calculate prin similitudini a scăzut ușor, lucru confirmat în intervalul de 1%–15% date alterate, unde doar un singur client era țintit de sistem pentru infecție. Pe de altă parte, medierea standard prin FedAvg s-a dovedit masiv instabilă sub un atac cu distribuție mare a impactului (în special la 40% și 50% date otrăvite). Metoda de apărare FoolsGold a reușit cu succes să atenueze efectele atacului de tip label-flipping în intervalul dominant de 20%–55% date compromise din populație, demonstrând că modelul poate fi protejat în mod fiabil prin setarea unei metode de agregare defensive corecte alături de parametrii corespunzători.

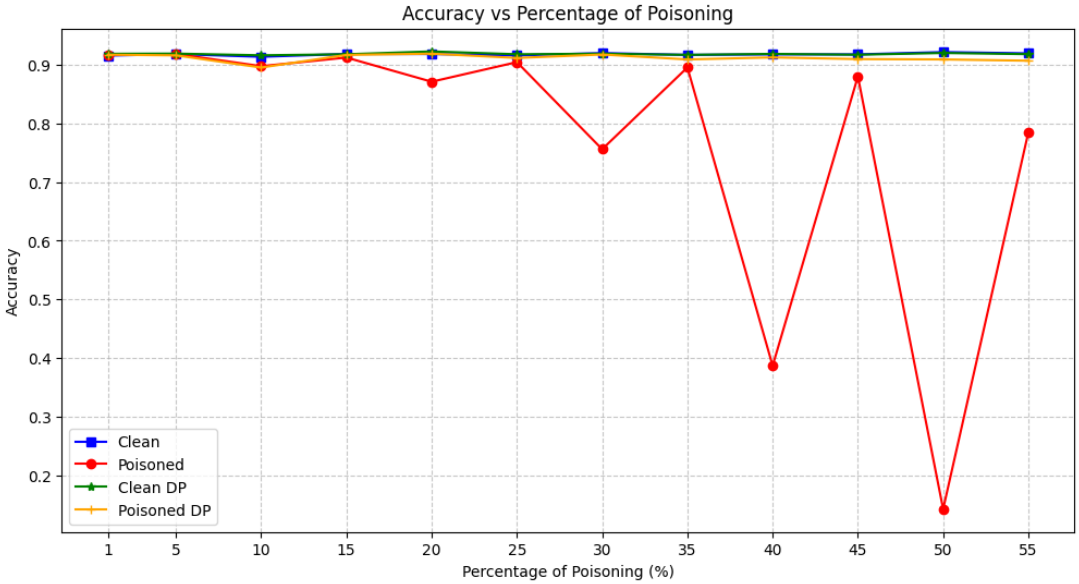

Figura 4.1: Rezultate Experimentale - Cazul 1

4.3 Studiu de Caz 2: Cea mai bună protecție pentru un atac specific

În acest studiu de caz se consideră un scenariu în care un cercetător dorește identificarea celui mai eficient mecanism de apărare pentru un atac fix de otrăvire a datelor. Obiectivul este de a compara mai multe strategii de protecție (agregare) în aceleași condiții experimentale. Spre

deosebire de testul anterior, arhitectura modelului, tipul atacului și intensitatea rămân complet constante.

4.3.1 Detalii practice

Același model ResNet18 antrenat pe CIFAR-10 a fost folosit ca referință. Rețeaua federată a constat dintr-un total de 10 clienți interconectați ($N = 10$). Pentru a simula o prezență malițioasă extrem de dăunătoare și coordonată, 3 dintre acești clienți ($M = 3$, reprezentând 30% din rețea) au fost setați să acționeze sistematic împotriva orchestratorului. Antrenamentul s-a desfășurat pe durata a 5 runde globale, iar atacul de tip Label-Flipping a vizat 100% din datele de antrenament locale ale nodurilor aservite atacatorului.

În aceste condiții dure și fixe, au fost realizate mai multe simulări, singura variabilă modificată fiind metoda de apărare de la nivelul serverului central. Mai exact, a fost analizată performanța a patru mecanisme principale: FoolsGold, Trimmed Mean, Krum și Norm Clipping.

4.3.2 Rezultate experimentale

Evaluările au evidențiat diferențe majore în eficacitatea metodelor de apărare împotriva acestui grad ridicat de compromitere (30% din dispozitive):

Acuratețea modelului pre-antrenat inițial (descărcat înainte de propagare) a fost calculată la aproximativ 0.7334, iar performanța curată de referință a modelului antrenat federat (fără atac, folosind exclusiv agragarea FedAvg normală) a atins rapid pragul de 0.8556. Analizând comportamentul sub tensiunea atacului de label-flipping, s-au obținut următoarele diferențe notabile:

- **FedAvg (fără protecție)** a servit drept variantă de subsol (baseline sub atac) și a generat o acuratețe finală de doar 0.5546. Aceasta reprezintă o scădere fatală de -0.1788 chiar și față de modelul inițial descărcat de pe internet și o scădere de -0.3010 față de media curată optimă, confirmând vulnerabilitatea critică a medierii ponderate clasice în fața unui atac coordonat din partea a o treime din rețea.

- **FoolsGold** s-a dovedit a fi clar cea mai eficientă metodă defensivă pe acest tip de configurație, atingând o acuratețe de 0.8464 în ciuda celor 3 agenți atacatori activi — o scădere microscopică de numai -0.0092 față de situația curată fără niciun atac. Analizând actualizările prin similaritate cosinus, orchestratorul a putut depista că toți clienții atacatori furnizează un set comun de caracteristici greșite și le-a redus drastic sau chiar suprimat aportul spre modelul global.

- **Norm Clipping** a obținut al doilea cel mai bun rezultat, cu o acuratețe finală menținută la nivelul de 0.7468 (reprezentând o creștere reziduală de $+0.0134$ comparativ cu modelul pre-antrenat inițial). Limitând magnitudinea permisă pentru norma vectorilor trimiși de clienți, mecanismul a oprit creșterea forțată a ponderilor infectate dinspre atacatori fără a fi nevoit să își reducă rețeaua total acceptată de conexiuni.

- **Trimmed Mean** a restrâns o acuratețe moderată la nivelul de 0.5874. Deși a adus o îmbunătățire utilă față de absența totală a apărării, eliminarea parametrilor statistici de la extemele superioare și inferioare nu a asigurat o barieră completă, efortul coordonat creand un grad mare de raportări eronate.

- **Krum** a eșuat și a oferit paradoxal cea mai slabă protecție din cadrul bateriei de teste, obținând o acuratețe de doar 0.5132 (situată chiar sub rezistența FedAvg simplu). Concepția logicii acestui algoritm alege un singur vector din rândul update-urilor clienților preluându-l pe cel ce are cea mai mică distanță euclidiană față de ceilalți. Cei 3 clienți malițioși cu pachete foarte similare, dar profund eronate s-au susținut teoretic reciproc și au ridicat exponențial riscul algoritmului de a califica doar o „încercare de atac" drepentru cândidat valid de încredere, corupând în final total mediul.

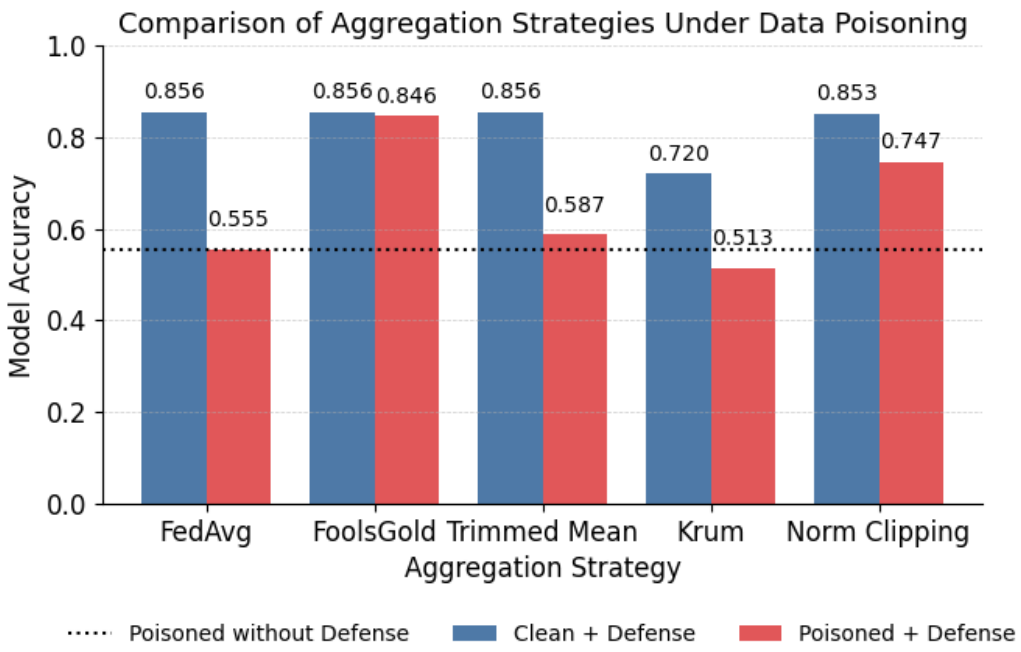

Figura 4.2: Rezultate Experimentale - Cazul 2

4.4 Interpretarea rezultatelor

Rezultatele celor două studii de caz indică faptul că eficiența unui mecanism de apărare nu poate fi evaluată izolat de structura atacului și de distribuția datelor între clienți. În primul studiu, pe măsură ce proporția datelor compromise crește, diferența dintre scenariul curat și cel atacat devine din ce în ce mai pronunțată, ceea ce confirmă faptul că intensitatea otrăvirii influențează direct capacitatea modelului global de a învăța reprezentări stabile. Totodată, apărarea bazată pe similaritatea actualizărilor, precum *FoolsGold*, devine mai eficientă atunci când atacatorii produc tipare corelate suficient de puternice pentru a fi identificate.

În al doilea studiu de caz, diferențele dintre mecanismele de apărare arată că nu toate metodele robuste răspund la fel în fața unui atac coordonat. *FoolsGold* se comportă foarte bine deoarece exploatează tocmai redundanța comportamentului malițios dintre clienții atacatori. *Norm Clipping* oferă un compromis rezonabil, reducând amplitudinea influenței malițioase fără a elimina complet efectul atacului. În schimb, metode precum *Trimmed Mean* și mai ales *Krum* devin mai vulnerabile atunci când ipotezele lor de funcționare sunt afectate de prezența simultană a unei distribuții Non-IID și a unor actualizări malițioase coerente între ele.

Dintr-o perspectivă mai largă, experimentele confirmă că problema nu constă doar în existența atacului, ci și în dificultatea separării actualizărilor legitime, dar heterogene, de cele efectiv

compromise. Această observație este importantă pentru proiectul de față, deoarece arată că platforma nu produce doar rapoarte descriptive, ci poate evidenția relații cauzale relevante între parametrii experimentali, alegerea apărării și stabilitatea modelului global.

4.5 Discuție comparativă cu literatura de specialitate

Pentru a consolida relevanța experimentală a rezultatelor obținute, acestea pot fi raportate la două lucrări importante din literatura de specialitate: studiul *Can You Really Backdoor Federated Learning?* [14], respectiv lucrarea de sinteză *Advances and Open Problems în Federated Learning* [6]. Prima dintre acestea are un caracter preponderent experimental și analizează eficiența unor atacuri de tip backdoor și a unor apărări precum *Norm Clipping*, în timp ce a doua oferă cadrul teoretic larg al învățării federate, discutând limitările mediilor Non-IID, vulnerabilitatea la atacuri bizantine și rolul estimărilor robuste în agregare.

Rezultatele din prezenta lucrare sunt compatibile cu observațiile lui Sun et al. [14] în două direcții. În primul rând, aceștia arată că performanța atacului depinde de fracția de clienți compromiși și de gradul de coordonare dintre ei. Aceeași concluzie se reflectă și în Studiul de Caz 1, unde degradarea modelului devine vizibil mai accentuată pe măsură ce proporția datelor compromise crește, iar apărarea bazată pe similaritatea actualizărilor devine mai eficientă când atacatorii sunt suficient de numeroși pentru a produce tipare corelate. În al doilea rând, studiul lor arată că metode precum *Norm Clipping* pot limita parțial efectele actualizărilor malițioase, fără a elimina însă complet riscul. Această observație este confirmată și în Studiul de Caz 2, unde *Norm Clipping* obține al doilea cel mai bun rezultat, dar rămâne sub performanța metodei *FoolsGold*.

În raport cu lucrarea lui Kairouz et al. [6], experimentele realizate validează în special două dificultăți structurale ale învățării federate. Prima este heterogenitatea datelor între clienți. Survey-ul subliniază că distribuțiile Non-IID afectează convergența și fac mai dificilă separarea variației legitime a actualizărilor de comportamentul malițios. Rezultatele obținute în cadrul platformei, pe distribuții dominate local de clase diferite, confirmă această dificultate prin variațiile mai mari dintre scenariile curate și atacate. A două dificultate ține de alegerea mecanismului de agregare: literatura indică faptul că nu există o metodă universal robustă pentru toate tipurile de atac și toate configurațiile federate. Exact acest aspect este evidențiat și de experimentele proprii, unde *FoolsGold*, *Norm Clipping*, *Trimmed Mean* și *Krum* au comportamente semnificativ diferite sub același atac de tip *Label-Flipping*.

În plus, paperul propriu pregătit pentru publicăre, *DPArena: A Platform for Reproducible Experiments on Data Poisoning în Federated Learning* [15], oferă o punte directă între observațiile din literatura anterioară și contribuția practică a acestei lucrări. Dacă studiul lui Sun et al. este centrat pe evaluarea atacurilor și a câtorva apărări într-un cadru experimental specific, iar lucrarea lui Kairouz et al. inventariază provocările deschise ale domeniului, DPArena transformă aceste nevoi într-un instrument software reproductibil. Platforma propusă permite tocmai testarea sistematică a relației dintre intensitatea otrăvirii, distribuția clienților malițioși și eficiența algoritmilor de agregare, reducând dependența de scripturi izolate și facilitând comparații reproductibile între scenarii.

Prin urmare, comparația cu aceste lucrări întărește validitatea experimentelor realizate: rezultatele nu apar ca observații izolate, ci se înscriu coerent în direcția raportată deja în literatura de specialitate. În același timp, prin automatizarea întregului flux experimental și prin suportul pentru funcții personalizate de agregare și poisoning, platforma dezvoltată extinde abordările anterioare dintr-o perspectivă inginerească, oferind un cadru practic pentru validarea continuă a noilor ipoteze de cercetare.

4.6 Limitări și validitatea rezultatelor

Deși rezultatele obținute sunt relevante pentru obiectivul lucrării, ele trebuie interpretate în limitele configurațiilor experimentale utilizate. În primul rând, experimentele au fost realizate pe un set de date de tip imagine și pe o arhitectură de model bine cunoscută, ceea ce înseamnă că concluziile nu pot fi transferate automat către toate tipurile de date sau către toate modelele utilizate în învățarea federată. În mod similar, tipurile de atac analizate sunt reprezentative, dar nu epuizează întregul spațiu al atacurilor de otrăvire posibile.

Din perspectiva validității interne, rezultatele sunt influențate de parametri precum distribuția Non-IID, numărul de clienți, numărul de runde, procentajul de date compromise și alegerea hiperparametrilor de antrenare. Platforma controlează aceste variabile și le documentează explicit, însă relația dintre ele rămâne una complexă. Prin urmare, rezultatele trebuie înțelese ca valide pentru configurațiile evaluate și ca argumente experimentale puternice, nu ca demonstrații universale pentru orice mediu federat.

Din perspectiva validității externe, contribuția principală a acestui capitol nu constă în generalizarea absolută a valorilor numerice, ci în identificarea unor tendințe robuste: creșterea intensității otrăvirii degradează performanța modelului, distribuțiile Non-IID complică detecția comportamentului malițios, iar diferitele mecanisme de agregare au comportamente semnificativ diferite în funcție de natura atacului. Aceste concluzii sunt suficient de consistente pentru a susține utilitatea platformei ca instrument de cercetare și analiză comparativă.

4.7 Reproductibilitatea experimentelor

Un avantaj important al platformei propuse constă în faptul că experimentele pot fi repetate într-o manieră controlată. Configurația fiecărei simulări este salvată împreună cu parametrii de antrenare, tipul atacului, mecanismul de agregare și artefactele generate în timpul execuției. Această persistență permite reluarea unor scenarii anterioare și compararea lor directă cu alte rulări realizate în condiții apropiate.

Reproductibilitatea este susținută și de separarea mediilor de execuție. Utilizarea unor medii Python dedicate, cu versiuni explicite de biblioteci, reduce variațiile introduse de dependențe incompatibile sau de actualizări necontrolate. În plus, fișierele standardizate generate la finalul simulării, precum `analysis.json`, `summary.txt` și `attack_info.json`, oferă un format stabil pentru analiză ulterioară, export și validare comparativă.

Din acest motiv, valoarea experimentală a platformei nu rezidă doar în rezultatele obținute în cadrul acestei lucrări, ci și în capacitatea ei de a susține experimente viitoare în condiții reproductibile. Acest aspect este deosebit de important într-un domeniu în care multe evaluări depind puternic de detalii de implementare greu de urmărit în absența unui cadru software unificat.

Capitolul 5:

Concluzii și dezvoltare ulterioară

Platforma prezentată în această lucrare reprezintă o contribuție în domeniul securității algoritmilor de învățare automată prin crearea unui mediu izolat, specific pentru testarea atacului de otrăvire a datelor. Aceasta a fost construită pentru a ajuta atât inginerii de securitate cibernetică, cât și inginerii de date care lucrează cu rețele neuronale deja antrenate să realizeze o baterie de teste cuprinzătoare înainte de a introduce aceste modele în producție, într-un mediu federat, în vederea îmbunătățirii rezilienței rețelei la acest tip de atac.

5.1 Starea Curentă

Proiectul prezentat în starea curentă este format din două părți. Prima parte este cea în care se construiește mediul de containere Docker, iar a doua este cea în care se construiește orchestratorul și se implementează etapele din jurul lui, prezentate în Figura 3.2. În implementarea actuală, platforma dispune de o interfață modernă, în care utilizatorul se înregistrează și își poate crea mai multe proiecte și teste (simulări) cu diferite moduri de configurare. La finalizarea fiecărei simulări, rezultatul este vizibil și poate fi salvat local sau poate fi comparat cu alte simulări. Platforma suportă antrenarea de rețele neuronale folosind biblioteca TensorFlow și permite realizarea tuturor etapelor prezentate în capitolul 3.

5.2 Dezvoltare Ulterioară

5.2.1 Limitări actuale

În forma sa actuală, platforma prezintă mai multe limitări care trebuie asumate explicit. În primul rând, suportul experimental este orientat în principal către seturi de date de tip imagine și către scenarii de antrenare compatibile cu structura șabloanelor definite în proiect. Din acest motiv, extinderea directă către date textuale, serii temporale sau scenarii multimodale necesită adaptări suplimentare ale infrastructurii.

În al doilea rând, scalabilitatea este condiționată de resursele hardware disponibile pe serverul de simulare. Numărul de clienți federați, dimensiunea modelelor și durata totală a experimentelor sunt limitate de memoria GPU, de spațiul de stocare și de politica de alocare concurentă a task-urilor. Această constrângere nu afectează validitatea arhitecturii, dar influențează amploarea experimentelor ce pot fi executate într-un interval rezonabil de timp.

O altă limitare privește dependența de servicii și resurse externe, precum sursele de modele pre-antrenate sau seturi de date descărcate din repozi-toare publice. În cazul indisponibilității acestora, unele scenarii nu mai pot fi reproduse identic. În mod similar, monitorizarea în timp real prin WebSocket și comunicarea dintre backend și orchestrator introduc puncte suplimentare de fragilitate în cazul conexiunilor instabile.

Nu în ultimul rând, mecanismul de extensibilitate prin funcții personalizate oferă flexibilitate ridicată, dar nu poate garanta automat corectitudinea științifică a algoritmilor definiți de utilizator. Platforma validează sintaxa, structura și anumite constrângeri de securitate, însă calitatea metodologică a funcțiilor integrate rămâne în responsabilitatea cercetătorului.

5.2.2 Direcții de dezvoltare

În procesul ulterior de dezvoltare, o direcție importantă este extinderea suportului pentru o varietate mai mare de scenarii experimentale și pentru o integrare mai flexibilă a mediilor de antrenare. De asemenea, se poate rafina implementarea anumitor atacuri, precum *Gaussian Noise*, prin utilizarea unor mecanisme adaptive dependente de distribuția actualizărilor locale sau de caracteristicile reale ale datelor compromise.

O altă direcție relevantă este creșterea robusteții infrastructurii de execuție. Aceasta include îmbunătățirea toleranței la erori de rețea, consolidarea politicilor de reluare a task-urilor întrerupte și extinderea mecanismelor de audit pentru reproducerea precisă a fiecărei simulări. În paralel, platforma poate evolua prin suport pentru seturi de date suplimentare, modele mai mari și scenarii cu mai mulți clienți federati.

O privire de ansamblu asupra problemelor deschise, limitărilor cunoscute și riscurilor tehnice identificate pentru platforma de simulare, ne conduc la următoarele neajunsuri: dimensiunea maximă acceptată pentru fișierele de model, limita de 100 de clienți federați impusă de constrângerile de memorie GPU, suportul exclusiv pentru seturi de date de tip imagine, dependența de disponibilitatea serviciilor externe (HuggingFace Hub) și riscul pierderii conexiunii WebSocket în timpul simulărilor de lungă durată. Aceste aspecte vor fi adresate prioritar în iterațiile viitoare ale platformei.

Bibliografie

Cărți

Articole Științifice

[6] Peter Kairouz et al., "Advances and Open Problems in Federated Learning", în *Foundations and Trends® in Machine Learning* 14.1-2 (2021), pp. 1–210, DOI: 10.1561/2200000083, URL: https://doi.org/10.1561/2200000083.

[8] Leslie Lamport, Robert Shostak și Marshall Pease, "The Byzantine Generals Problem", în *ACM Transactions on Programming Languages and Systems (TOPLAS)* 4.3 (1982), pp. 382–401.

[13] Minghong Fang et al., "Local Model Poisoning Attacks to Byzantine-Robust Federated Learning", în *arXiv preprint arXiv:1911.11815* (2019).

[14] Ziteng Sun et al., "Can You Really Backdoor Federated Learning?", în *arXiv preprint arXiv:1911.07963* (2019), URL: https://arxiv.org/abs/1911.07963.

[7] Keith Bonawitz, Hubert Eichner și Wolfgang Grieskamp, "Towards Federated Learning at Scale: System Design", în *Proceedings of the 2nd SysML Conference*, Palo Alto, CA, USA, 2019.

[9] Peva Blanchard et al., "Machine Learning with Adversaries: Byzantine Tolerant Gradient Descent", în *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 118–128.

[10] Lingjiao Chen et al., "DRACO: Byzantine-Resilient Distributed Training via Redundant Gradients", în *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018.

[11] Dong Yin et al., "Byzantine-Robust Distributed Learning: Towards Optimal Statistical Rates", în *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019.

Referințe Online și Diverse

[1] Directoratul Național de Securitate Cibernetică (DNSC), *Buletin de Indicatori Statistici și Tendințe de Securitate Cibernetică - H1 2025*, https://www.dnsc.ro/vezi/document/buletin-de-indicatori-statistici-si-tendinte-de-securitate-cibernetica-h1-2025, Accesat în 2025-11-27, 2025.

[2] Future of Life Institute, *AI Act Overview - Future of Life Institute (30 May 2024)*, https://artificialintelligenceact.eu/wp-content/uploads/2024/11/Future-of-Life-InstituteAI-Act-overview-30-May-2024.pdf, Accessed: 2025-11-27, mai 2024.

[3] IBM, *IBM Federated Learning API Documentation*, https://ibmfl-api-docs.res.ibm.com/index.html, Accesat în 2025-11-28, 2025.

[4] IBM, *IBM Federated Learning Library - GitHub Repository*, https://github.com/IBM/federated-learning-lib/tree/main, Accesat în 2025-11-28, 2025.

[5] IBM, *AI vs. Machine Learning vs. Deep Learning vs. Neural Networks*, https://www.ibm.com/think/topics/ai-vs-machine-learning-vs-deep-learning-vs-neural-networks, Accesat în 2025-11-28, 2025.

[12] Clement Fung, Chris J. M. Yoon și Ivan Beschastnikh, *Mitigating Sybils in Federated Learning Poisoning*, 2020, arXiv: 1808.04866 [cs.LG].

[15] Tudor Lepădatu, Andrei Brînzea și Iulian Aciobăniţei, *DPArena: A Platform for Reproducible Experiments on Data Poisoning in Federated Learning*, Software tool article, compiled March 15, 2026, mar. 2026.